

EDA

Selene Schmittling, PhD

Exploratory Data Analysis (EDA) in R

Learning Outcomes

By the end of this workshop, you will be able to

- Load data into a dataframe
- Assess general characteristics of your data (e.g. data types, column names)
- Generate descriptive statistics
- Quantify missingness and outliers
- Visualize univariate, bivariate and multivariate data.
- Identify potential relationships of interest which could be explored using modeling

We will use a dataset which has been handed to us by Barbara who works at NC State University. She has asked us to determine the following:

1. What columns are in the data and what are their values?
2. Are there any problems with the data? Outliers? Missing Values? Data type issues?
3. What variance to you see in the individual values?
4. What is the covariance between variables in the dataset?
5. Does covariance between any variables lead to an interesting question?

Barbara would like reporting and visualization of the data. We have no additional information about the data except the following: units used in the data are inches, pounds, dollars and years.

Install tidyverse

Let's start by installing, if needed, and loading the package of packages, `tidyverse`.

```
# If tidyverse isn't installed, install it
if (!require(tidyverse)) {
  install.packages("tidyverse")
}

# load tidyverse
library(tidyverse)
```

Load Data

```
# Create variable to hold url to raw data in github
data.url <- "../data/EDA_Data_File_1.csv"

# read in data
data <- read.csv(data.url)

# Look at first few rows of data
head(data)
```

	Name	Species	Breed		Prior.Position
1	Pixel	Turtle	Eastern Box		Rock Impersonator
2	Teddy	Bird	Cockatiel	Mirror	Interaction Specialist
3	Wallace Whitfield Riddick	Human			
4	Rocky	Tortoise	Red-footed		Slow Motion Life Coach
5	Tuffy I	Dog	Tamaskan		Child Actor
6	Daisy	Bird	Cockatoo		Alarm Clock Supervisor
	Prior.Employer	Interview.Date	Hire.Year	Hire.Date	Termination.Year
1	No Moves, LLC	1997-08-25	NA		NA
2	Homegoods	1985-10-03	NA		NA
3		1899-02-17	1899	1899-02-23	NA
4	Best You, Inc.	1974-09-05	NA		NA
5	Abrams Artists Agency	2010-10-11	2010	2010-10-13	NA
6	The Clock Depot	2003-01-10	NA		NA
	Termination.Date	Termination.Reason	Post.Employment.Position	Nick.Name	
1				Gizmo	
2					

46		Petco	2009-01-13	NA
47			1960-02-17	1960 1960-02-23
48	NC State University Police Department		1960-10-21	NA
49	Spit Shine Beauty Boutique		1951-05-23	NA
50			2010-10-18	2010 2010-10-25

Termination.Year	Termination.Date	Termination.Reason
------------------	------------------	--------------------

41	NA	
42	NA	
43	NA	
44	NA	
45	NA	
46	NA	
47	NA	Escaped
48	NA	
49	NA	
50	NA	Puppy Mill Puppy

Post.Employment.Position	Nick.Name	Temperment	Fur.Color	Age	Height
	Crinkles	sweet		9.2	6.10
		cranky	gray	12.2	13.20
	Crinkles	distracted	buff	8.4	7.32
		distracted		14.2	3.25
	Pooky	distracted		8.6	20.00
	Doc	bewildered	multi-color	8.6	5.70
Art Thief		devilishly clever	multi-color	7.4	31.00
	Crinkles	patient	black	9.0	25.30
		patient	brown	8.5	36.77
		social	brown	11.6	28.50

Weight	Offer.Sent	Starting.Salary	Biter	Hired
41	0.12	0	NA	0
42	12.95	1	10478	1
43	0.88	1	10068	0
44	0.0003	1	9658	0
45	0.92	0	NA	0
46	0.28	0	NA	0
47	120.73	1	10424	0
48	72	0	NA	0
49	149.99	0	NA	0
50	75.0	1	10349	0

Summarize Data

Shape & Type

The `glimpse()` function returns information about our data frame (the data structure that holds our data) including:

- the shape of our data, e.g. the number of rows and columns;
- column names, data type (`<chr>`, `<int>`, `<dbl>`) and a sample of values

```
glimpse(data)
```

```
Rows: 50
Columns: 22
$ Name           <chr> "Pixel", "Teddy", "Wallace Whitfield Riddick"~
$ Species        <chr> "Turtle", "Bird", "Human", "Tortoise", "Dog",~
$ Breed          <chr> "Eastern Box", "Cockatiel", "", "Red-footed",~
$ Prior.Position <chr> "Rock Impersonator", "Mirror Interaction Spec~
$ Prior.Employer <chr> "No Moves, LLC", "Homegoods", "", "Best You, ~
$ Interview.Date <chr> "1997-08-25", "1985-10-03", "1899-02-17", "19~
$ Hire.Year      <int> NA, NA, 1899, NA, 2010, NA, NA, NA, 1966, NA,~
$ Hire.Date      <chr> "", "", "1899-02-23", "", "2010-10-13", "", "~
$ Termination.Year <int> NA, NA, NA, NA, NA, NA, NA, NA, 1970, NA, NA,~
$ Termination.Date <chr> "", "", "", "", "", "", "", "", "1970-02-~
27",~
$ Termination.Reason <chr> "", "", "", "", "Retired", "", "", "", "Retir~
$ Post.Employment.Position <chr> "", "", "", "", "Mattress Sales", "", "", "",~
$ Nick.Name       <chr> "Gizmo", "", "", "Fruitloops", "Wave", "Profe~
$ Temperment      <chr> "patient", "mouthy", "patient", "devilishly c~
$ Fur.Color       <chr> "", "", "cream", "", "brown", "", "brown", ""~
$ Age             <dbl> 11.6, 8.4, 5.0, 8.4, 10.8, 10.6, 9.6, 13.0, 9~
$ Height          <dbl> 5.44, 12.22, 34.50, 12.85, 24.30, 17.76, 10.7~
$ Weight          <chr> "1.49", "0.215", "40.0", "25.11", "70.0", "1.~
$ Offer.Sent      <int> 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, ~
$ Starting.Salary <int> NA, NA, 9481, NA, 9965, NA, NA, NA, 9313, NA,~
$ Biter           <int> 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, ~
$ Hired           <int> 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, ~
```

In the notebook this looks messy, let's use our history file to call this in the console and see if it looks better. To do this:

- To the right, in the **Environment Pane**, click on the **History** tab.

- Scroll all the way to the bottom, the last command shown will be the last one you ran, either in the console or in your notebook.
- Click on `glimpse(data)` to select it.
- Just below the **History** tab, click on “To Console”.
- Look in your **Console** window. Magic!! Click in the console window and hit enter to run the command.

Question: What do you notice about the data?

POSSIBLE ANSWERS:

1. There are 50 rows (observations) and 22 columns (variables)
 2. There is missing data.
 3. There is a mix of character and numeric data (`<int>` and `<dbl>`)
-

Values

We can use the `summary()` function to get information about each of our variables.

```
summary(data)
```

Name		Species		Breed		Prior.Position		Prior.Employer	
Length	:50	Length	:50	Length	:50	Length	:50	Length	:50
N.unique	:50	N.unique	:23	N.unique	:45	N.unique	:42	N.unique	:40
N.blank	: 0	N.blank	: 0	N.blank	: 1	N.blank	: 9	N.blank	: 9
Min.nchar:	3	Min.nchar:	3	Min.nchar:	0	Min.nchar:	0	Min.nchar:	0
Max.nchar:	25	Max.nchar:	9	Max.nchar:	23	Max.nchar:	32	Max.nchar:	38

Interview.Date		Hire.Year		Hire.Date		Termination.Year	
Length	:50	Min.	:1899	Length	:50	Min.	:1946
N.unique	:50	1st Qu.:	:1932	N.unique	:13	1st Qu.:	:1956
N.blank	: 0	Median	:1963	N.blank	:38	Median	:1964
Min.nchar:	10	Mean	:1964	Min.nchar:	0	Mean	:1974

Max.nchar:10	3rd Qu.:2010	Max.nchar:10	3rd Qu.:1982
	Max. :2021		Max. :2020
	NAs :38		NAs :46

Termination.Date	Termination.Reason	Post.Employment.Position	Nick.Name
Length :50	Length :50	Length :50	Length :50
N.unique : 5	N.unique : 6	N.unique : 4	N.unique :15
N.blank :46	N.blank :43	N.blank :47	N.blank :29
Min.nchar: 0	Min.nchar: 0	Min.nchar: 0	Min.nchar: 0
Max.nchar:10	Max.nchar:16	Max.nchar:14	Max.nchar:10

Temperment	Fur.Color	Age	Height	Weight
Length :50	Length :50	Min. : 5.00	Min. : 1.25	Length :50
N.unique :12	N.unique : 8	1st Qu.: 9.10	1st Qu.: 7.66	N.unique :50
N.blank : 1	N.blank :17	Median :10.20	Median :13.22	N.blank : 0
Min.nchar: 0	Min.nchar: 0	Mean :10.32	Mean :19.28	Min.nchar: 2
Max.nchar:17	Max.nchar:11	3rd Qu.:11.60	3rd Qu.:25.98	Max.nchar: 8
		Max. :14.70	Max. :77.34	

Offer.Sent	Starting.Salary	Biter	Hired
Min. :0.00	Min. : 9313	Min. :0.00	Min. :0.00
1st Qu.:0.00	1st Qu.: 9572	1st Qu.:0.00	1st Qu.:0.00
Median :0.00	Median : 9904	Median :0.00	Median :0.00
Mean :0.34	Mean : 9863	Mean :0.08	Mean :0.24
3rd Qu.:1.00	3rd Qu.:10086	3rd Qu.:0.00	3rd Qu.:0.00
Max. :1.00	Max. :10478	Max. :1.00	Max. :1.00
	NAs :33		

Again, the console will work better for output. The summary provides the following information:

Character Data

- Length: max # of characters)
- N.unique: number of unique values
- N.blank: how many blank (missing? unknown?)
- Min.nchar: minimum length
- Max.nchar: maximum length

Numeric Data

- Min: minimum value

- 1st Qu.: first quartile value
- Median: median
- Mean: mean
- 3rd Qu: third quartile value
- Max: maximum value

A NOTE ABOUT BOOLEAN DATA (0/1)

Fields that contain boolean values (0/1) show interesting characteristics in a summary:

- Min will be 0 and Max will be 1.
 - The mean will represent the percentage of rows for which the value is set to 1
- For instance, The mean for Hired is 0.24 which means 24% of the rows have Hired set to 1.

Question: What do you notice about the summarized data?

POSSIBLE ANSWERS:

1. Some of the character data have 50 unique values.
2. A few of the numeric fields have a lot of NAs.
3. Weight is a character field.
4. Dates are treated as character data, but years are not.

Next steps:

- identify variables for which we think missing data might be problematic
- assess whether the data type associated with a column make sense for the data stored in it

Missing Data

We can explore missing data exclusively by using the `dplyr` package which is part of `tidyverse`. We want to extract the number of blanks in character data and the number of zeros or NAs in numeric data.

We will use the R pipe, `|>`. This will allow us to take the output of one command and pass it as the argument to the next. The `<-` operator assigns the value to a variable.

```
# Quantify the number of NAs and blanks in the data
missing_counts_dplyr <- data |>
  summarise(across(everything(), ~sum(is.na(.) | . == "")))

print(missing_counts_dplyr)
```

```
  Name Species Breed Prior.Position Prior.Employer Interview.Date Hire.Year
1      0      0      1              9              9              0         38
  Hire.Date Termination.Year Termination.Date Termination.Reason
1      38             46             46             43
  Post.Employment.Position Nick.Name Temperment Fur.Color Age Height Weight
1             47          29             1          17      0      0      0
  Offer.Sent Starting.Salary Biter Hired
1           0             33      0      0
```

- `across()` applies a transformation across multiple columns in a data frame.
- `everything()` selects all variables
- `~sum(is.na(.) | . == "")` essentially sums either blanks or `is.na()` in the data where `.` represents a column. Functions used in this way (where the variable is not defined) is call an anonymous function.

Changing Data Type(s)

Text to Numeric

From our summary we saw that `Weight` is character data. This is not an appropriate data type for a numeric value. Let's fix that.

```
# Check values before changing
summary(data$Weight)
```

```
Length N.unique N.blank Min.nchar Max.nchar
      50       50        0         2         8
```

```
# Convert Weight to numeric value
data$Weight <- as.numeric(data$Weight)
```

Warning: NAs introduced by coercion

```
# Check converted data
glimpse(data$Weight)
```

```
num [1:50] 1.49 0.215 40 25.11 70 ...
```

```
summary(data$Weight)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
0.0003	0.9100	10.9000	73.7352	72.0000	1076.8200	1

WAIT!!

Where did the NA come from? Let's compare the character data to the numeric. Find where the NA is and then find it's location in the character data. What is the problem? Hmm.

We can fix this using the following code.

```
# identify the row that has the NA value for weight
idx <- which(is.na(data$Weight))

# Replace NA with the correct numeric value
data$Weight[idx] <- 1000.11 # What if more than 1 row/value needed to be fixed?

# Check the Weight vector
data$Weight
```

[1]	1.4900	0.2150	40.0000	25.1100	70.0000	1.6600	10.2000
[8]	0.9100	30.5000	2.8100	3.7100	57.6800	8.5000	1.8000
[15]	90.0000	0.0800	0.1320	850.2800	55.0000	100.1500	0.9300
[22]	51.0000	38.0900	68.5000	0.0730	3.1800	0.0090	72.0000
[29]	148.9000	0.3500	112.5000	1000.1100	145.0100	1076.8200	80.0000
[36]	0.0040	10.7000	0.0100	10.9000	10.9500	0.1200	12.9500
[43]	0.8800	0.0003	0.9200	0.2800	120.7300	72.0000	149.9900
[50]	75.0000						

Text to Date

Another conversion we probably want to make is to convert text dates to a date format. Since we only want to convert a Date and not time information, we can use the `as.Date()` function.

```
# Convert Interview.Date to date format
data$Interview.Date <- as.Date(data$Interview.Date,
                              format = "%Y-%m-%d")

# Check converted data
glimpse(data$Interview.Date)
```

```
Date[1:50], format: "1997-08-25" "1985-10-03" "1899-02-17" "1974-09-05" "2010-
10-11" ...
```

```
summary(data$Interview.Date)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
"1899-02-17"	"1958-09-19"	"1982-03-27"	"1979-02-21"	"2004-01-10"	"2024-09-09"

Question: How can we tell that our Interview.Date has been converted to a date?

ANSWERS:

The summary shows numeric attributes, e.g. Min., 1st Qu., Median, Mean, 3rd Qu., and Max.

TRY IT!

Using the code above as a template, convert Hire.Date and Termination.Date from text to a date.

```
# Hire.Date
data$Hire.Date <- as.Date(data$Hire.Date,
                          format = "%Y-%m-%d")

# Check converted data
glimpse(data$Hire.Date)
```

```
Date[1:50], format: NA NA "1899-02-23" NA "2010-10-13" NA NA NA "1966-03-07" NA NA NA NA ...
```

```
summary(data$Hire.Date)
```

```
      Min.      1st Qu.      Median      Mean      3rd Qu.      Max.
"1899-02-23" "1932-11-16" "1963-03-01" "1964-09-04" "2010-10-16" "2021-11-17"
      NAs
      "38"
```

```
# Termination.Date
data$Termination.Date <- as.Date(data$Termination.Date,
                                format = "%Y-%m-%d")

# Check converted data
glimpse(data$Termination.Date)
```

```
Date[1:50], format: NA NA NA NA NA NA NA NA "1970-02-27" NA NA NA NA NA NA NA NA NA NA ...
```

```
summary(data$Termination.Date)
```

```
      Min.      1st Qu.      Median      Mean      3rd Qu.      Max.
"1940-05-17" "1954-08-25" "1964-10-12" "1972-09-12" "1982-10-31" "2020-11-09"
      NAs
      "46"
```

Boolean Flags to Factor

There are three boolean fields in our data: `Offer.sent`, `Biter` and `Hired` boolean data frequently represent flags, which are categorical variables in practice. Factors let you associate a level and a label to a categorical variable. Converting a categorical variable to a factor can simplify analysis and visualization in R.

```
# Offer.Sent
data$Offer.Sent <- factor(data$Offer.Sent,
                          levels = c(0,1),
                          labels = c("Offer Not Sent",
                                     "Offer Sent"))

# Check converted data
glimpse(data$Offer.Sent)
```

Factor w/ 2 levels "Offer Not Sent",...: 1 1 2 1 2 1 1 1 2 1 ...

```
summary(data$Offer.Sent)
```

Offer Not Sent	Offer Sent
33	17

Questions:

1. How can we find out what data type `Offer.Sent` is?
2. What data type is it?
3. Is **factor** a data type? (Hard question)
4. Can we do math on this field?

ANSWERS:

1. You can use `typeof()` to see what data type `Offer.Sent` has.
2. `typeof()` returns .
3. Technically, a factor is a data object, we can see this by running `class(data$Offer.Sent)`. This command returns “factor” which indicates the class of the object.
4. We can issues `sum(data$Offer.Sent)` to check if R sees this as a numeric value.

```
# 1. and 2. Use typeof() to see what data type the field is
typeof(data$Offer.Sent)
```

```
[1] "integer"
```

```
# 3. We can use class() to see that Offer.Sent is and object of class "factor"
class(data$Offer.Sent)
```

```
[1] "factor"
```

```
# 4. We can use sum() to test whether we can do math on a factor Uncomment the line below to
# sum(data$Offer.Sent)

# We can use as.numeric() to force R to generate a sum. However, what does it mean?
sum(as.numeric(data$Offer.Sent))
```

[1] 67

```
# maybe this is more helpful
table(data$Offer.Sent)
```

Offer Not Sent	Offer Sent
33	17

TRY IT!

Using the code above as a template, convert `Biter` and `Hired` from `int` to a factor.

```
# Biter
data$Biter <- factor(data$Biter,
                     levels = c(0,1),
                     labels = c("Doesn't Bite",
                               "Bites"))
# Check converted data
glimpse(data$Biter)
```

Factor w/ 2 levels "Doesn't Bite",...: 1 2 1 2 1 1 2 1 1 1 ...

```
summary(data$Biter)
```

Doesn't Bite	Bites
46	4

```
# Hired
data$Hired <- factor(data$Hired,
                     levels = c(0,1),
                     labels = c("Not Hired",
                               "Hired"))
# Check converted data
glimpse(data$Hired)
```

Factor w/ 2 levels "Not Hired","Hired": 1 1 2 1 2 1 1 1 2 1 ...

```
summary(data$Hired)
```

Not Hired	Hired
38	12

Explore Character Data

We have many fields that store character data. In this section we'll explore ways to get more information about their contents. To do this, we first need to identify which columns in our data are character columns.

```
# identify character fields in our data
char.fields <- data |>
  select(where(is.character)) |> # a smaller df with character columns
  names()
print(char.fields)
```

```
[1] "Name"           "Species"
[3] "Breed"          "Prior.Position"
[5] "Prior.Employer" "Termination.Reason"
[7] "Post.Employment.Position" "Nick.Name"
[9] "Temperment"     "Fur.Color"
```

NOTE: This is the first time we have used the `tidyverse` pipe, `|>`. The pipe allows us to take the results of issuing a command on one line and pass the results to the command on the next. The code above takes `data` and passes it to the `select(where(is.character))` command. We then pass the output of the `select()` statement to the `names()` command. The results are stored in the `char.fields` variable.

We can look at the summary for a character fields and review how many blanks and unique values each has.

If a column has a lot of blanks, we might want to assess whether we need to get that data or not. The number of unique values will give us an idea of whether the field represents groups (few blanks and a limited number of unique values) or individuals (few blanks and a lot of unique values). Columns with a lot of unique values will be harder, if not impossible, to visualize.

The following code will create a table of character variables, the number of unique values and the number of blanks each variable has and then sorts the table.

```
# Simplest way: call summary on the subset of character columns
summary(data[,char.fields])
```

Name		Species		Breed		Prior.Position		Prior.Employer	
Length	:50	Length	:50	Length	:50	Length	:50	Length	:50
N.unique	:50	N.unique	:23	N.unique	:45	N.unique	:42	N.unique	:40
N.blank	: 0	N.blank	: 0	N.blank	: 1	N.blank	: 9	N.blank	: 9
Min.nchar:	3	Min.nchar:	3	Min.nchar:	0	Min.nchar:	0	Min.nchar:	0
Max.nchar:	25	Max.nchar:	9	Max.nchar:	23	Max.nchar:	32	Max.nchar:	38
Termination.Reason		Post.Employment.Position		Nick.Name		Temperment			
Length	:50	Length	:50	Length	:50	Length	:50	Length	:50
N.unique	: 6	N.unique	: 4	N.unique	:15	N.unique	:12	N.unique	:12
N.blank	:43	N.blank	:47	N.blank	:29	N.blank	: 1	N.blank	: 1
Min.nchar:	0	Min.nchar:	0	Min.nchar:	0	Min.nchar:	0	Min.nchar:	0
Max.nchar:	16	Max.nchar:	14	Max.nchar:	10	Max.nchar:	17	Max.nchar:	17
Fur.Color									
Length	:50								
N.unique	: 8								
N.blank	:17								
Min.nchar:	0								
Max.nchar:	11								

```
# More complex way. Get only the data we want:
for (c in char.fields) {
  print(c) # print column name
  print(summary(c)[c(2,3)]) # print N.unique and N.blank only
}
```

```
[1] "Name"
N.unique N.blank
      1      0
[1] "Species"
N.unique N.blank
      1      0
[1] "Breed"
N.unique N.blank
      1      0
[1] "Prior.Position"
N.unique N.blank
      1      0
[1] "Prior.Employer"
```

```

N.unique  N.blank
      1      0
[1] "Termination.Reason"
N.unique  N.blank
      1      0
[1] "Post.Employment.Position"
N.unique  N.blank
      1      0
[1] "Nick.Name"
N.unique  N.blank
      1      0
[1] "Temperment"
N.unique  N.blank
      1      0
[1] "Fur.Color"
N.unique  N.blank
      1      0

```

Questions:

1. What observations can we make about our character data?

POSSIBLE ANSWERS:

1. `Post.Employment.Position`, `Termination.Reason` and `Fur.Color` all have a small number of unique values but a large number of blanks.
2. `Nick.Name` has more blanks than values. It has 21 non-blanks which have 15 unique values.
3. `Temperment` has only 1 blank and 12 unique values.
4. `Species`, `Prior.Employer`, `Prior Position`, and `Breed` have low numbers of missing values but many non-unique values.
5. `Name` has no blanks and 50 unique values. For this data set, `Name` could be used as a unique identifier for a row.

For fields with a low to moderate number of unique values we can look at the unique values to get a sense of what they represent. To see the unique values we can use the `table()` command.

```
table(data$Post.Employment.Position)
```

	Art Thief	Mattress Sales	Traveling Zoo
47	1	1	1

TRY IT!

Using the code above as a template, look at the unique values for `Termination.Reason`, `Fur.Color`, `Nick.Name`, and `Temperment`.

```
# Termination Reason
```

```
table(data$Termination.Reason)
```

	Escaped	New Job	Passed Away
43	1	1	2
Puppy Mill Puppy	Retired		
1	2		

```
# Fur Color
```

```
table(data$Fur.Color)
```

	black	brown	buff	cream	gray
17	4	9	6	1	4
multi-color	white				
4	5				

```
# Nick Name
```

```
table(data$Nick.Name)
```

	Boo Boo	Crinkles	Doc	Fritz	Fruitloops	Gizmo
29	1	5	1	1	1	2
Hulk	Nyx	Pooky	Professor	Spock	Spuds	Wave
1	2	1	1	1	2	1
Yoda						
1						

```
# Temperment
table(data$Temperment)
```

	bewildered	calm	cranky
1	4	1	4
devilishly clever	distracted	frightened	mouthy
5	5	3	3
patient	quiet	social	sweet
10	3	7	4

For `Prior.Employer`, we can use a different method to explore unique values.

```
data |>
  distinct(Prior.Employer, .keep_all = TRUE) |>
  select(Prior.Employer)
```

	Prior.Employer
1	No Moves, LLC
2	Homegoods
3	
4	Best You, Inc.
5	Abrams Artists Agency
6	The Clock Depot
7	Walmart
8	US Navy
9	Tyson Foods
10	Hillside Civil Engineering
11	Raleigh Fire Department
12	Ducky Jewelry & Gemstones
13	Harris Teeter
14	Independent Contractor
15	Oustanding in Your Field
16	Artfully Coiffed
17	Pawsh & Co.
18	Philadelphia Zoo
19	Beak & Scatter, LLC
20	Snake Snack

21	Faculty Family
22	ACME Food Storage
23	Publix
24	Giggle
25	Drool Software
26	Contract to Hire, Inc.
27	Mrs. Smith's Garden
28	Knock it Down and Start Over, Inc.
29	New Clothes R Us
30	Laser Dog Security Agency
31	Outdoor Provision Company
32	REI
33	Petra & Holum
34	Beak, Claws, Paws & Reflect Counseling
35	Snooze Brothers
36	Spike's Security & Alarms
37	Chewy
38	Petco
39	NC State University Police Department
40	Spit Shine Beauty Boutique

TRY IT!

Using the code above as a template, look at the unique values for `Prior.Position`.

```
# Prior Position
data |>
  distinct(Prior.Position, .keep_all = TRUE) |>
  select(Prior.Position)
```

	Prior.Position
1	Rock Impersonator
2	Mirror Interaction Specialist
3	
4	Slow Motion Life Coach
5	Child Actor
6	Alarm Clock Supervisor
7	Late-Shift Announcements
8	Stealth Operations Specialist

9	Egg Production Supervisor
10	Burrow Engineer
11	Firehouse Morale Officer
12	Gemologist
13	Lettuce Taste Tester
14	Escape Artist
15	Cricket Procurement Specialist
16	Mane Styling Consultant
17	Chief Barketing Officer
18	Zoo
19	Seed Dispersal Engineer
20	Mouse Inspector
21	Pet
22	Treat Hoarder-in-Chief
23	Carrot Quality Control Associate
24	Web Infrastructure Designer
25	Slobber Consultant
26	Glass Cleaner
27	Carrot Acquisition Manager
28	Demolition Expert
29	Mud Puddle Inspector
30	Bubble Production Engineer
31	Laser Pointer Technician
32	Sunbeam Coordinator
33	Climbing Instructor
34	Senior Box Quality Analyst
35	Snuggle Specialist
36	Lead Nap Coordinator
37	Nocturnal Noise Maker
38	Flake Food Appraiser
39	Heat Lamp Tester
40	Wheel Operator
41	Paw-liceman
42	Fluff Consultant

Visualize Data

Univariate Data

Continuous

Appropriate visualizations for univariate continuous data include histograms and boxplots. Histograms give us information about the shape and spread of our data, showing us gaps, and modality. Boxplots, on the other hand give us a visual statistical summary of the data.

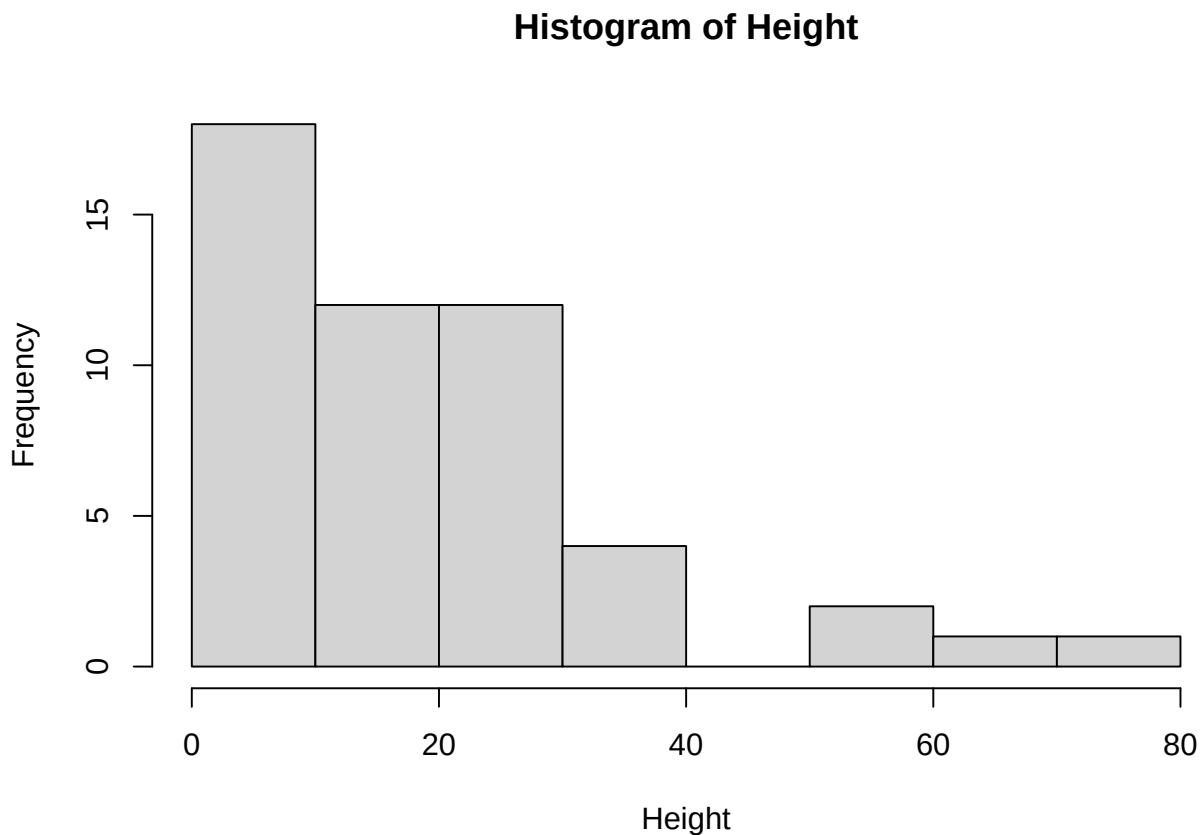
The fields in our data that are continuous are:

- Age
- Height
- Weight
- Starting.Salary

Let's compare the histogram and boxplot for Height.

Histogram

```
# make a histogram of Height
hist(data$Height,
      xlab = "Height",
      main = "Histogram of Height")
```



Question:

What observations can we make from our histogram?

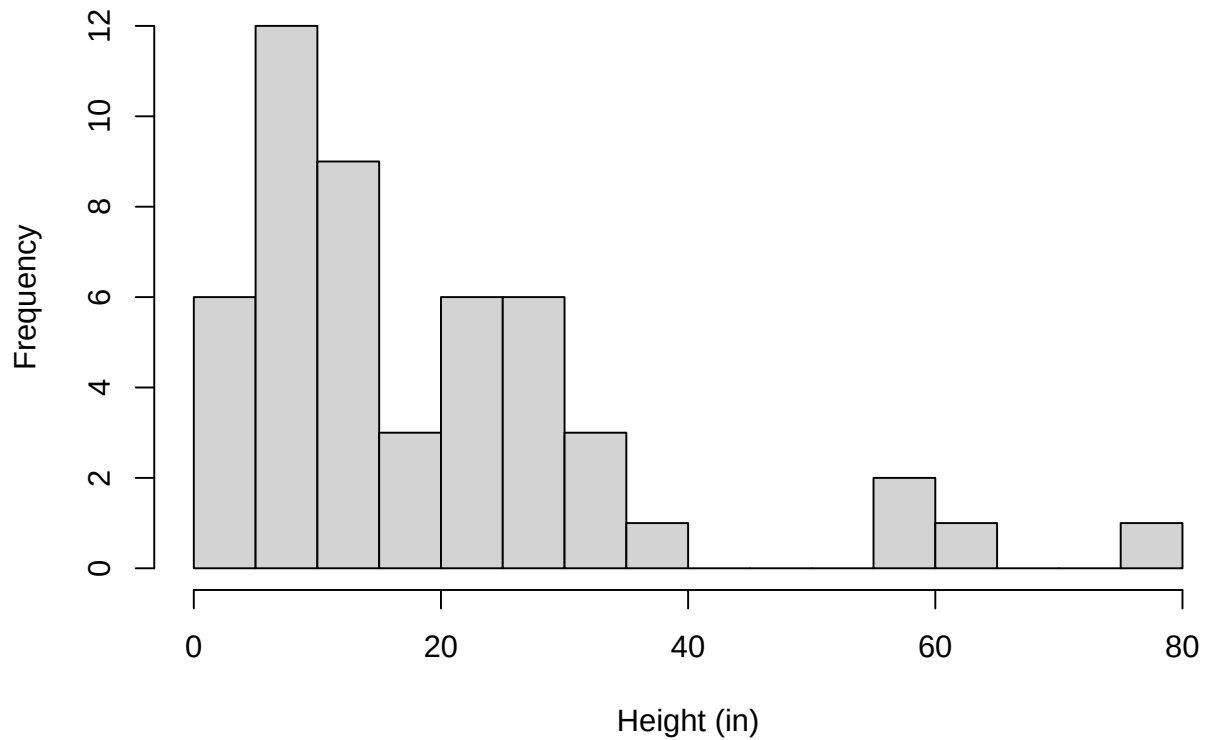
POSSIBLE ANSWERS:

1. Most of the heights occur below 10 inches.
 2. No heights occur between 40 and 50 inches
 3. A small number of weights are > 50 inches
 4. The maximum height is 80 inches.
-

Remember that histograms can be misleading if the number of bins or breaks are not enough. Let's increase the number of bins in our histogram of Height.

```
hist(data$Height,  
      breaks = 25, # this parameter lets us control the number of breaks/bins  
      xlab = "Height (in)",  
      main = "Histogram of Height"  
)
```

Histogram of Height



Question:

What observations can we make from our histogram with more bins?

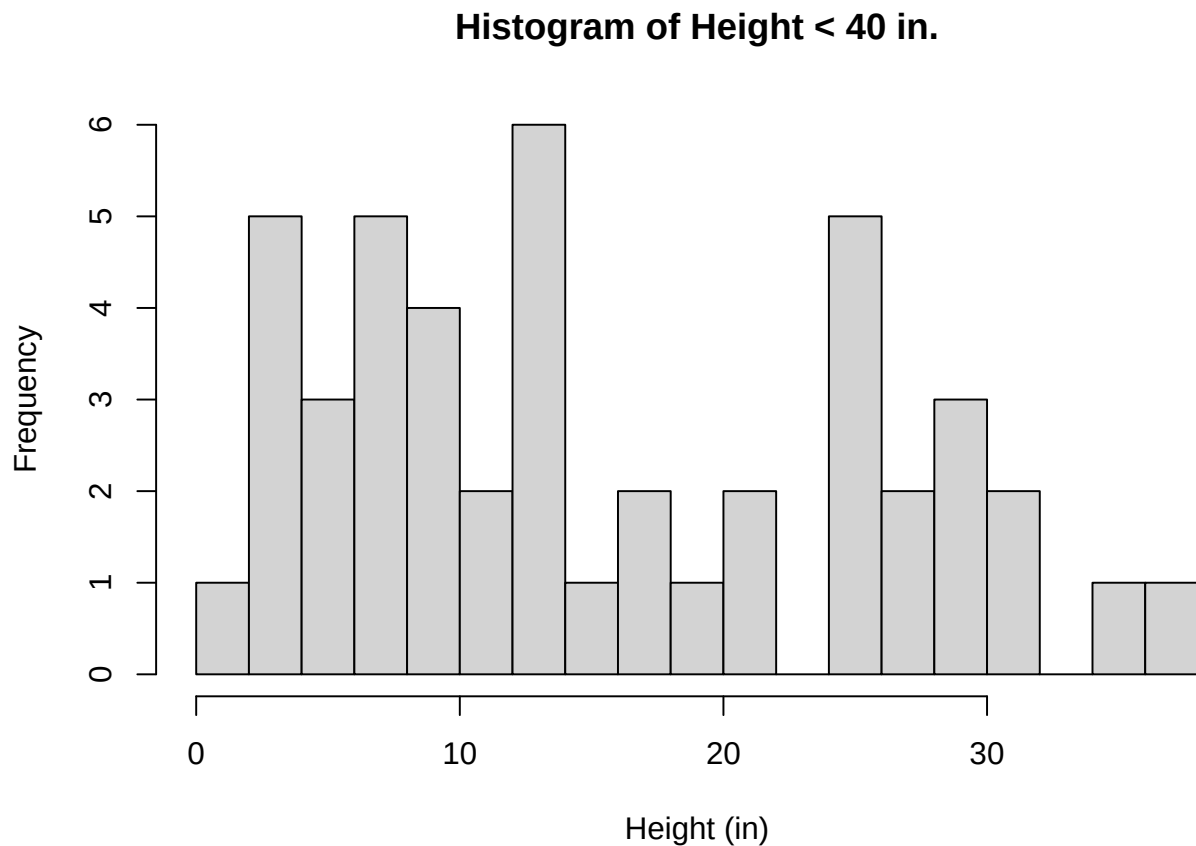
POSSIBLE ANSWERS:

1. There is more variation in $\text{Height} < 40$ than is clear from the default histogram.
2. The most observations occur around 5 in.
3. We can now see a gap between the # of observations around 60 vs. 80 inches.

We can investigate the distribution of just the observations with height < 40 .

```
height.subset <- data |>
  filter(Height <= 40)

hist(height.subset$Height,
      xlab = "Height (in)",
      main = "Histogram of Height < 40 in.",
      breaks = 15)
```



This gives us a better picture of how height is distributed below 40 inches. We might want to extract a little more data about the values above 40. Since these do not constitute a large group we can easily extract some information that might help us determine the nature of these observations

```
data |>
  filter(Height > 40) |>
  select('Name', 'Species', 'Breed', 'Height')
```

Name	Species	Breed	Height
------	---------	-------	--------

1	Duke	Horse	Arabian	57.73
2	Banjo	Dog	Saint Bernard	77.34
3	Draco	Horse	Morgan	59.00
4	Peanut	Horse	Quarter Horse	60.20

Well, this is interesting. While the values for Duke, Draco and Peanut are reasonable since they are horses, we do not expect Banjo to be taller than a horse! We should check into this value.

Next, let's compare the histogram to the boxplot.

Boxplot

Recall that a boxplot provides us with the following information:

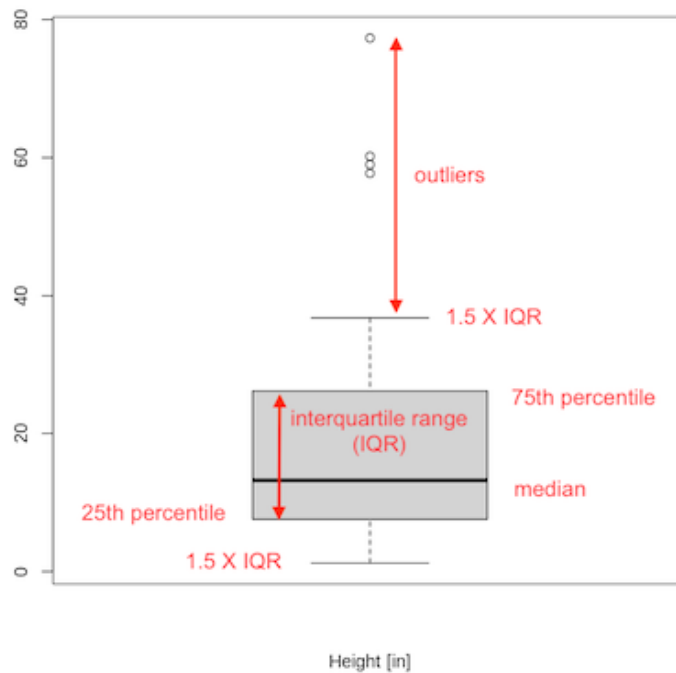
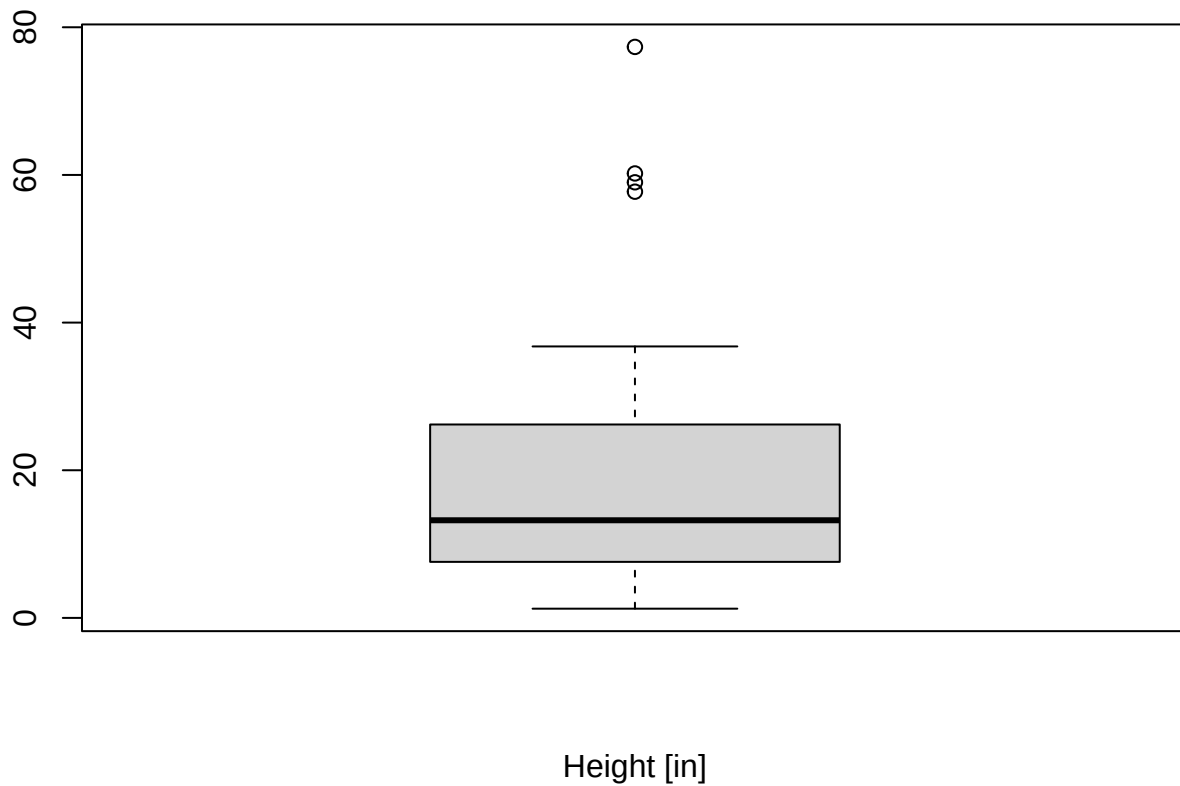


Figure 1: Boxplot features

Let's generate a boxplot for Height.

```
boxplot(data$Height,
        xlab = 'Height [in]')
```



Question:

What observations can we make from our boxplot?

POSSIBLE ANSWERS:

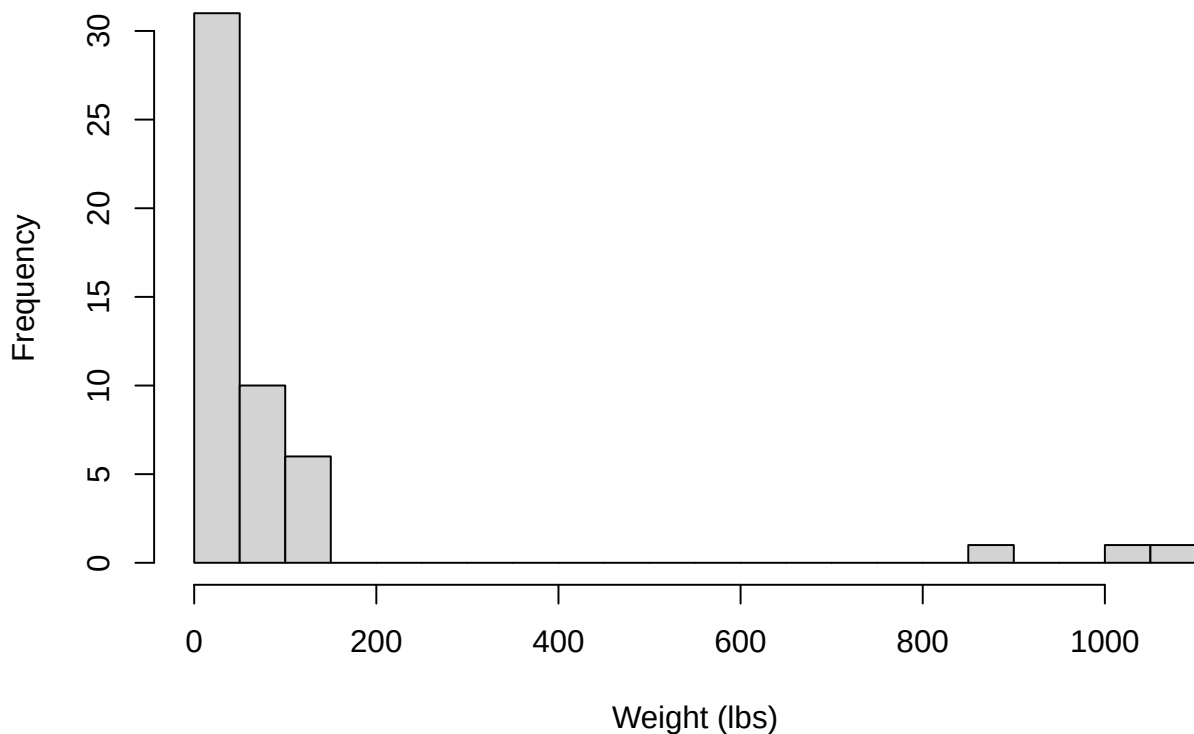
1. Four outliers occur above 50 inches.
 2. Heights are right tailed or left-skewed.
 3. 50% of the data points occur below ~ 15 inches.
 4. No outliers occur in the lower region.
-

TRY IT!

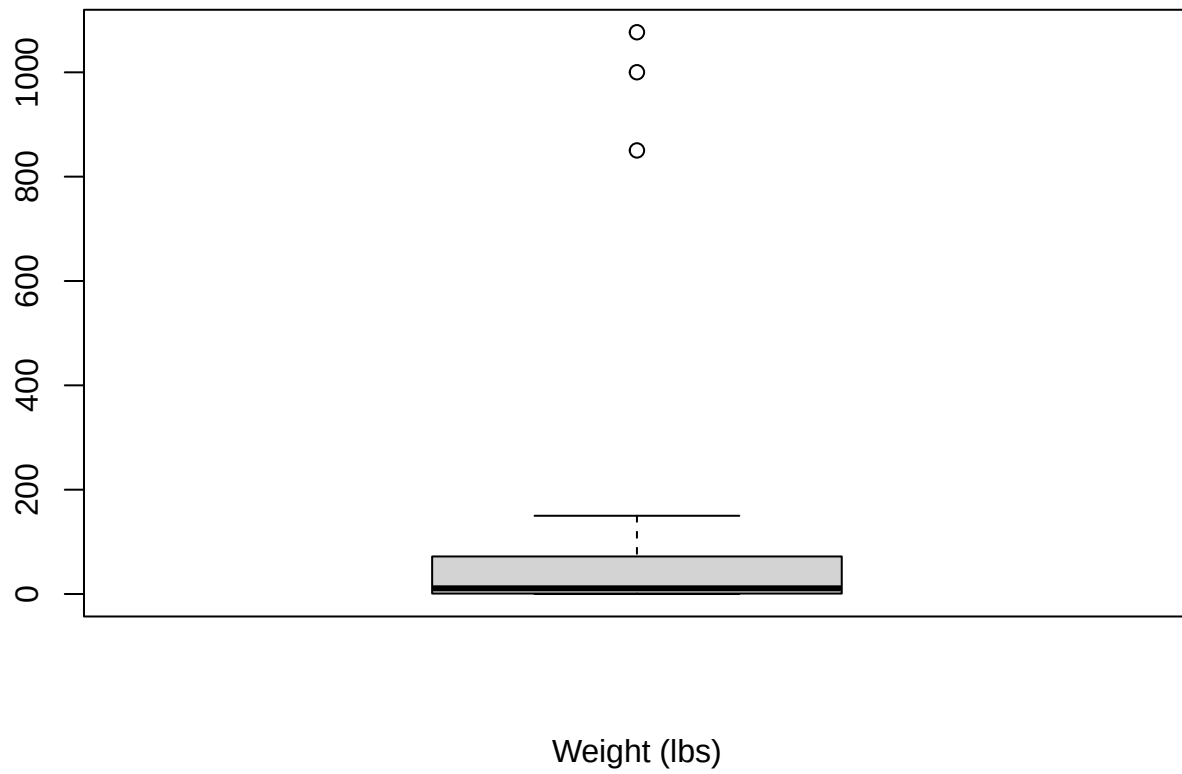
Using the code provided in the histogram and boxplot sections. Visually explore the values for **Weight**, **Age** and **Starting.Salary**. If necessary “zoom in” to a range of data to get more information.

```
# Weight - Histogram  
hist(data$Weight,  
      xlab = "Weight (lbs)",  
      main = "Histogram of Weight",  
      breaks = 25)
```

Histogram of Weight

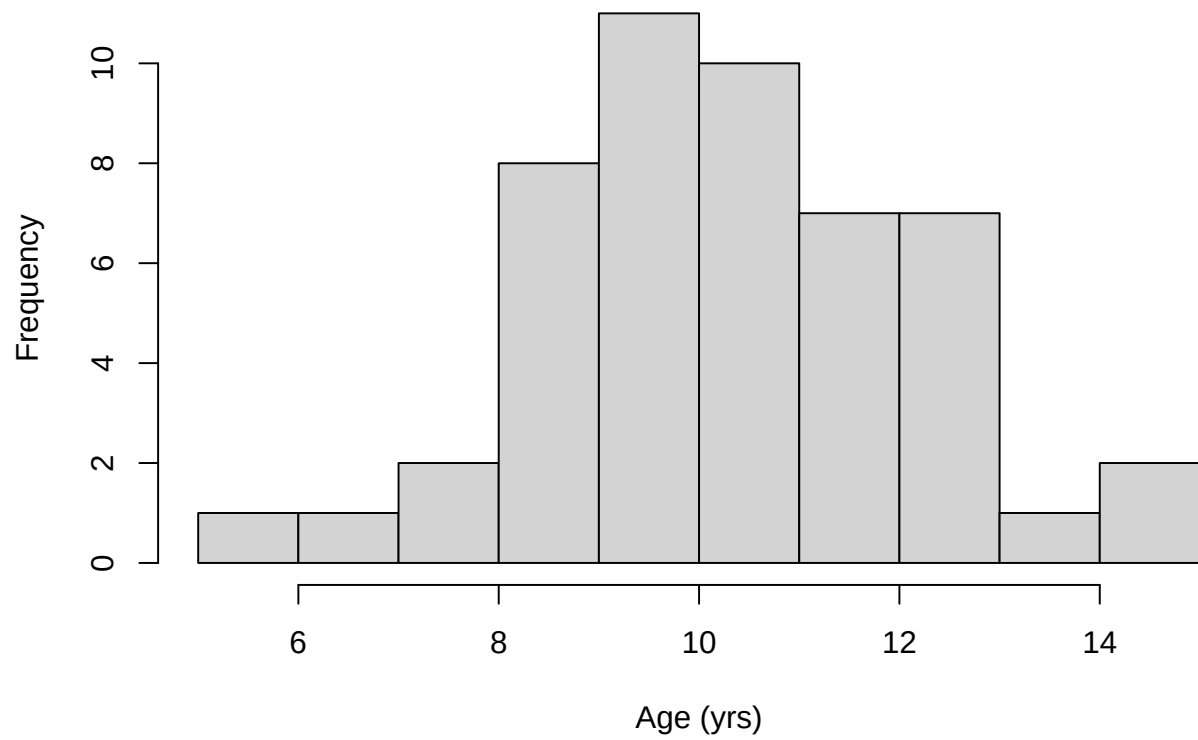


```
# Weight - Boxplot  
boxplot(data$Weight,  
        xlab = "Weight (lbs)")
```

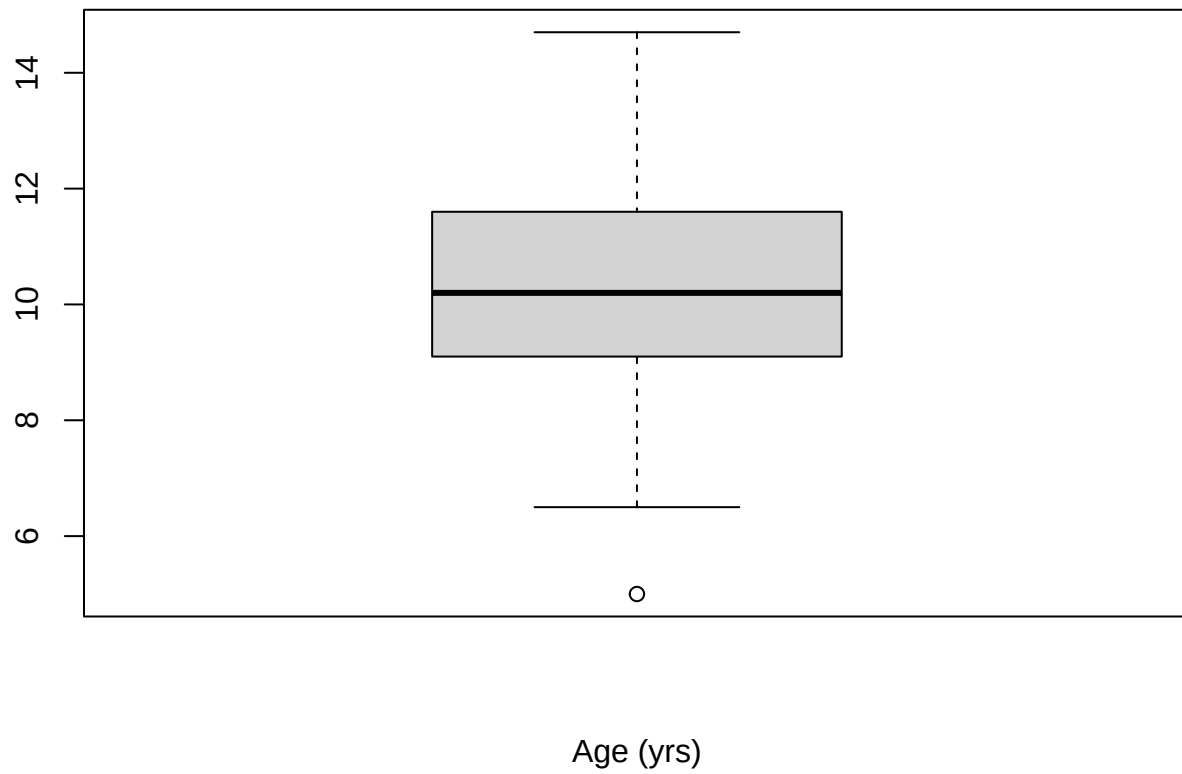


```
# Age - Histogram  
hist(data$Age,  
      xlab = "Age (yrs)",  
      main = "Histogram of Age")
```

Histogram of Age

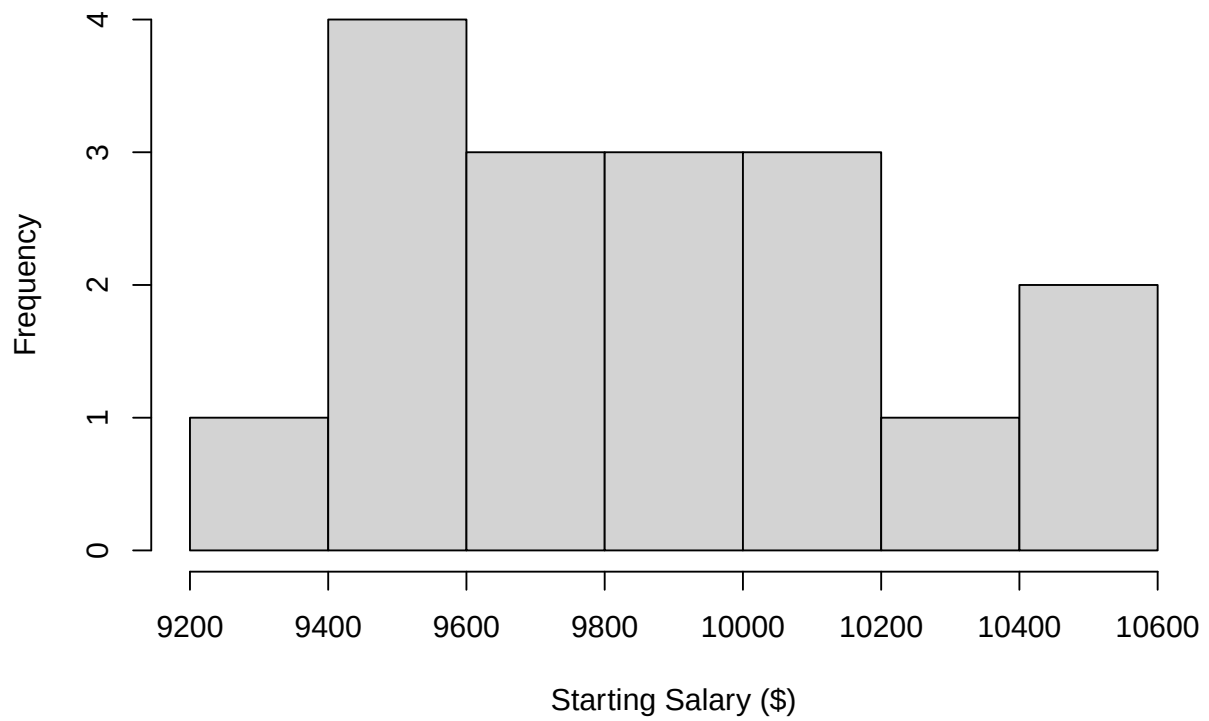


```
# Age - Boxplot  
boxplot(data$Age,  
        xlab = 'Age (yrs)')
```

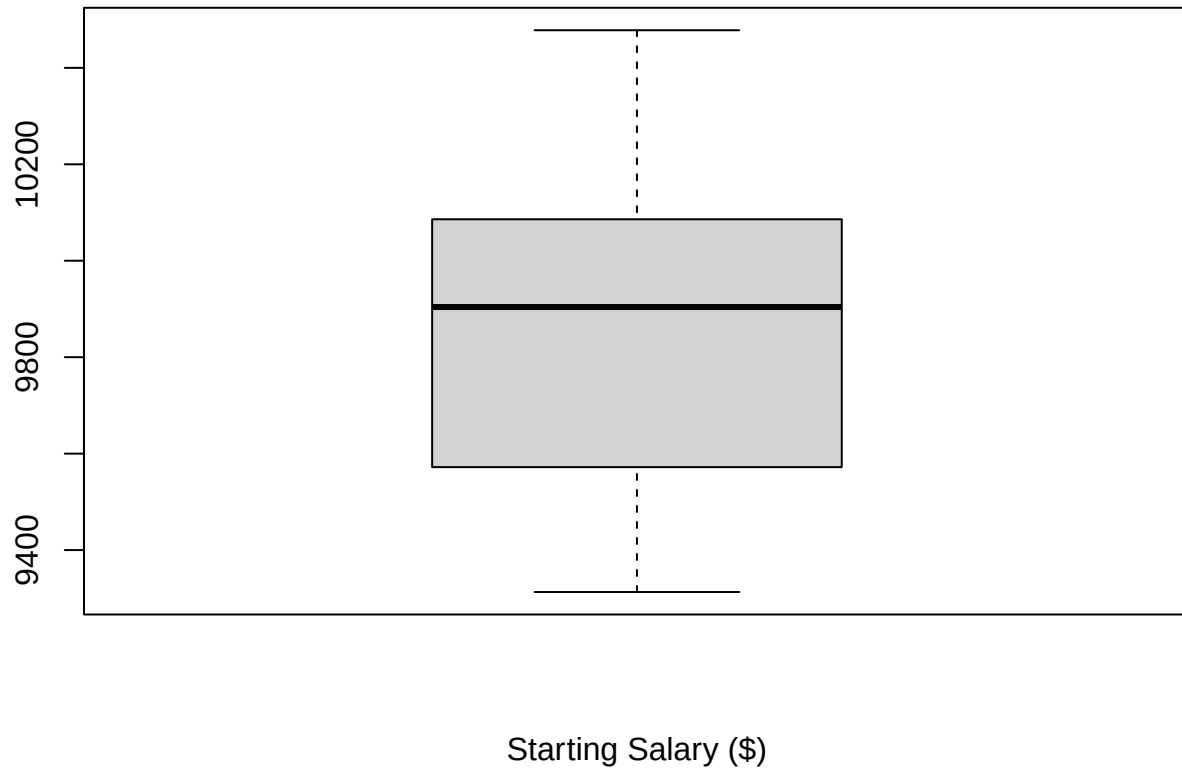


```
# Starting Salary - Histogram  
hist(data$Starting.Salary,  
      xlab = "Starting Salary ($)",  
      main = "Histogram of Starting Salary")
```

Histogram of Starting Salary



```
# Starting Salary - Boxplot  
boxplot(data$Starting.Salary,  
        xlab = 'Starting Salary ($)')
```



Discrete or Categorical

We use bar plots for univariate categorical data. The following columns in our data frame are categorical and in order of number of unique values:

1. Post.Employment.Position
2. Termination.Reason
3. Fur.Color
4. Temperment
5. Nick Name
6. Species
7. Prior.Employer
8. Prior.Position
9. Breed
10. Name

In addition we have our three variables that we converted to Factors: `Offer.Sent`, `Biter`, and `Hired`. These can be visualized the same way as other categorical variables.

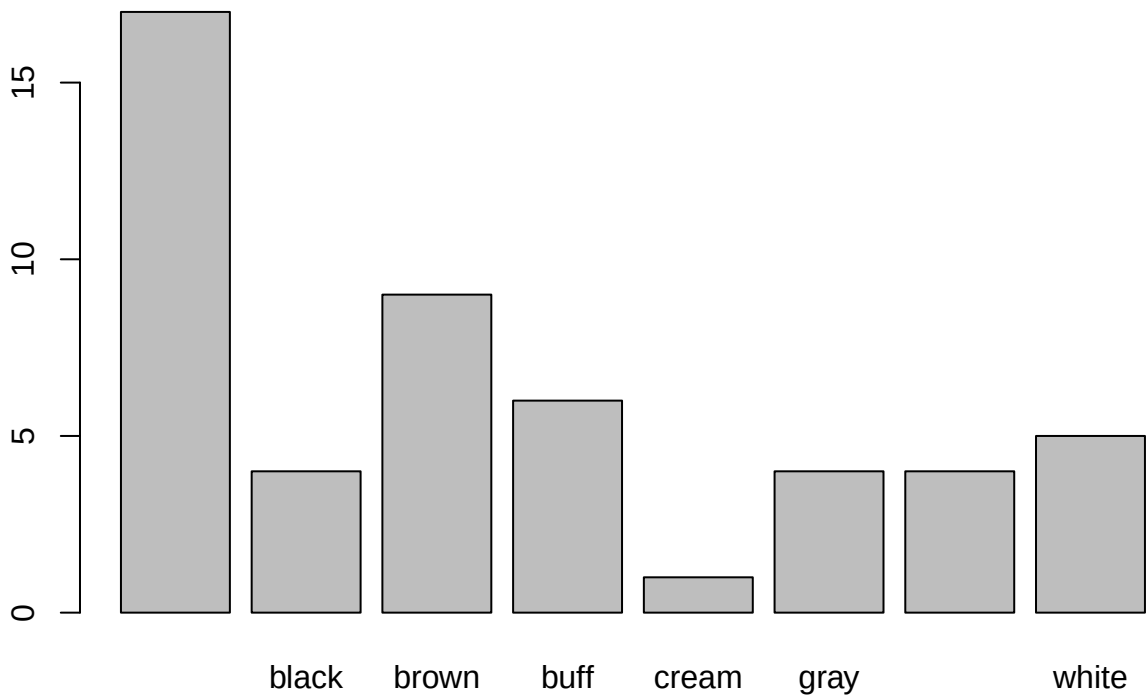
We'll start with columns that don't have a lot of unique values. We won't bother making plots of `Nick.Name` or `Name` since they are labels for the data. Let's start with `Fur.Color`.

NOTE: In order to create a bar plot, we need to generate the counts for each unique value in the column. We will use the `table()` function to provide that for us.

```
# Generate counts
cnts <- table(data$Fur.Color)
cnts
```

```
      black      brown      buff      cream      gray
multi-color      4      9      6      1      4
      17
      4      5
      white
```

```
# Generate bar plot
barplot(cnts)
```



There are some tweaks we can make to this which include adjusting the plot margins and flipping it so that we can see all the labels (notice the last bar is missing a label).

```
# Improve bar plot of Fur.Color
```

```
# Provide a label for blanks
```

```
names(cnts)
```

```
[1] ""          "black"      "brown"      "buff"       "cream"
[6] "gray"      "multi-color" "white"
```

```
names(cnts)[1] <- "blank"
```

```
# change borders on plot by modifying
```

```
# the mar parameter
```

```
par(mar = c(4,7,1,1))
```

```
# mar = c(bottom, left, top, right)
```

```
# default par(mar = c(5.1, 4.1, 4.1, 2.1))
```

```
# unit in lines of text
```

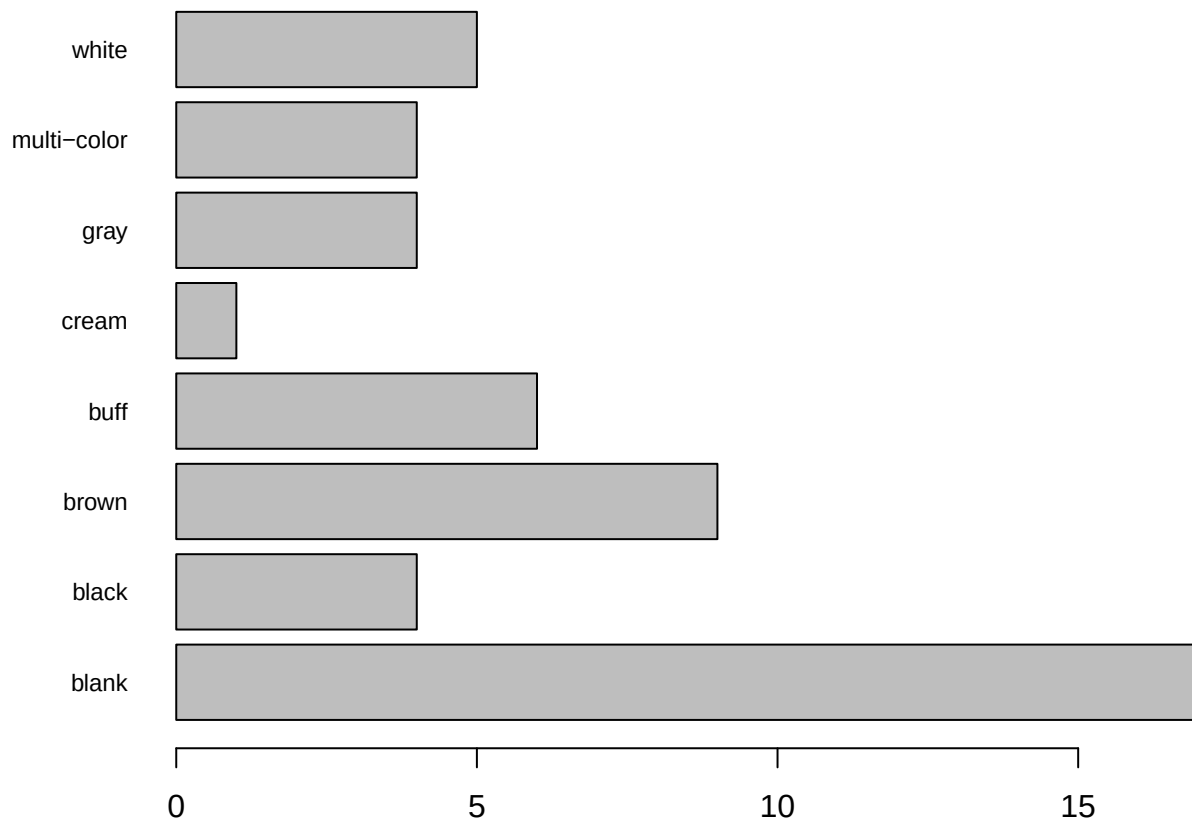
```
# Generate bar plot
```

```
barplot(cnts,
```

```
  horiz = TRUE,
```

```
  las = 1, # controls the orientation of the text wrt the axis (0-3)
```

```
  cex.names = .75) # makes the font size 75% of default size
```



```
# reset mar()
par(mar = c(5.1, 4.1, 4.1, 2.1))
```

A final improvement would be to sort the counts from smallest to largest.

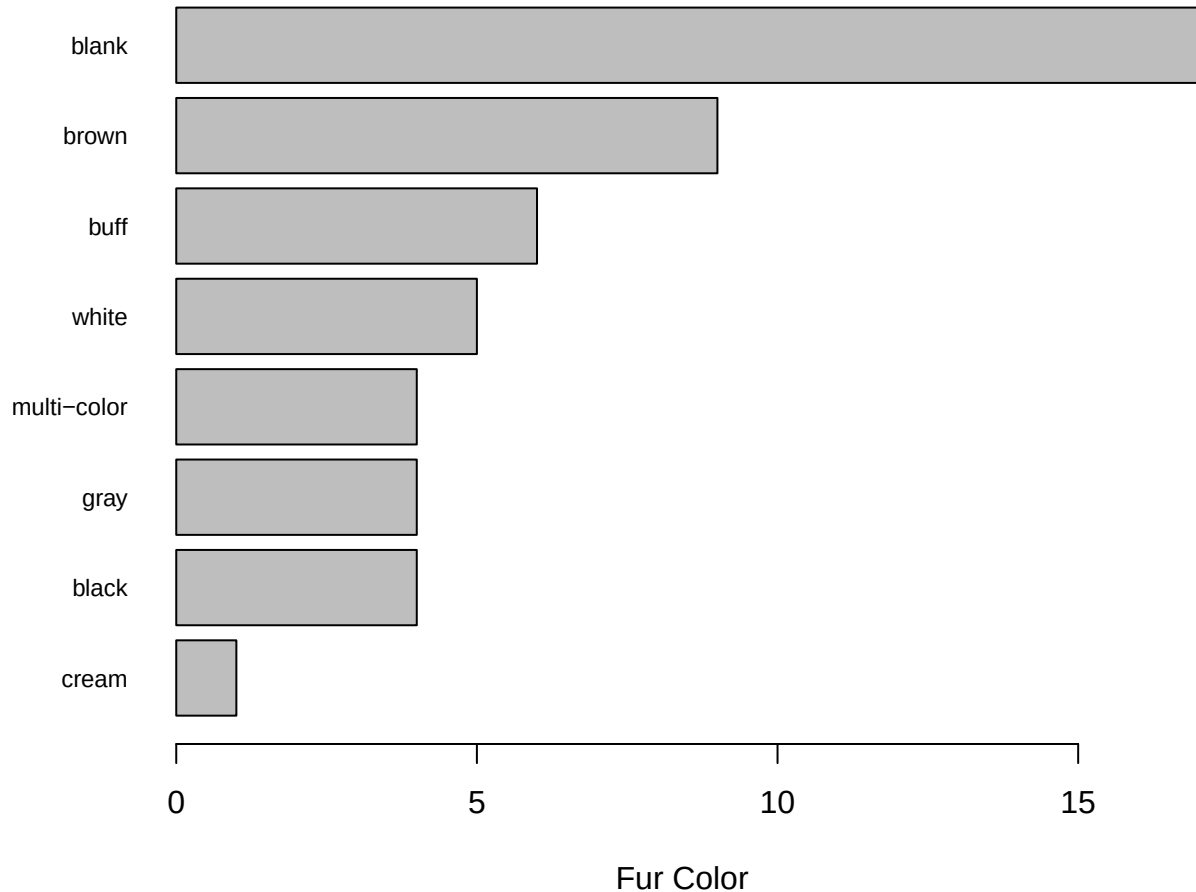
```
# Group data by breed and create counts
val.counts <- data |>
  group_by(Fur.Color) |>
  count() |>
  arrange(n)

val.counts$Fur.Color[8] <- "blank"

# Set mar()
par(mar = c(4,7,1,1))

# Generate bar plot
barplot(val.counts$n,
        names.arg = val.counts$Fur.Color,
```

```
xlab="Fur Color",  
horiz=TRUE,  
las = 1,  
cex.names = .75)
```



```
# reset mar()  
par(mar = c(5.1, 4.1, 4.1, 2.1))
```

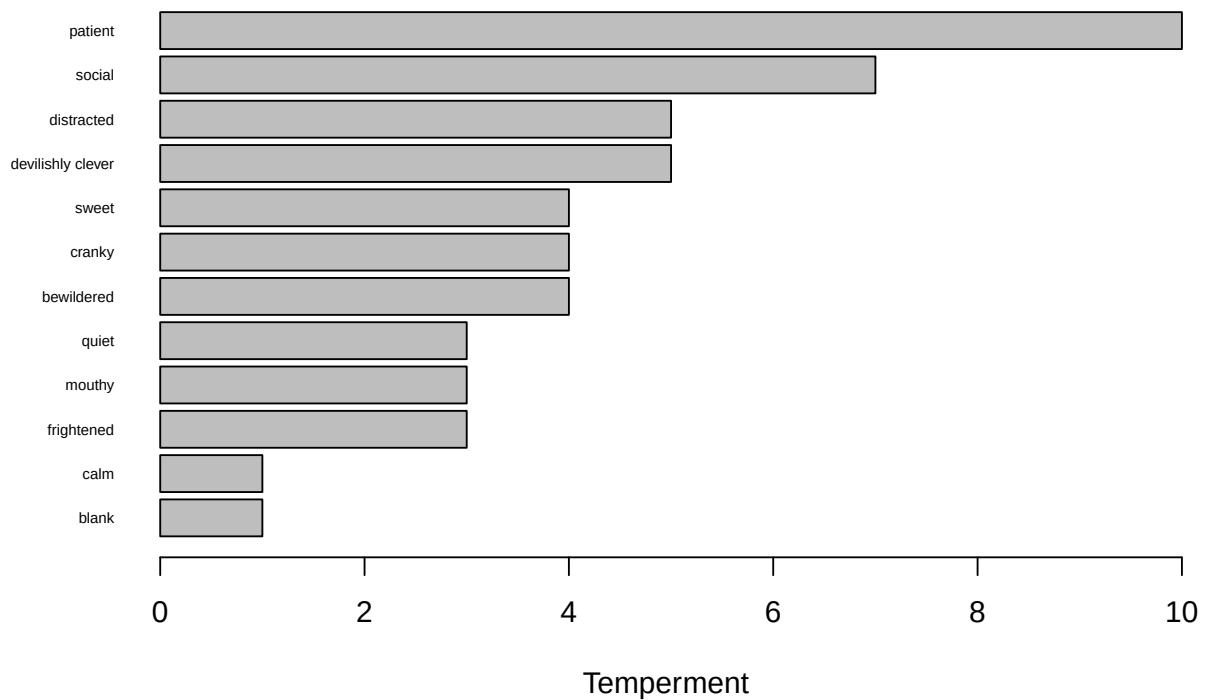
TRY IT!

Using the code above make some bar plots for other categorical fields in our data. Start with the basic code and then try some of the more advanced tricks we introduced. Remember to make a table of counts first!!

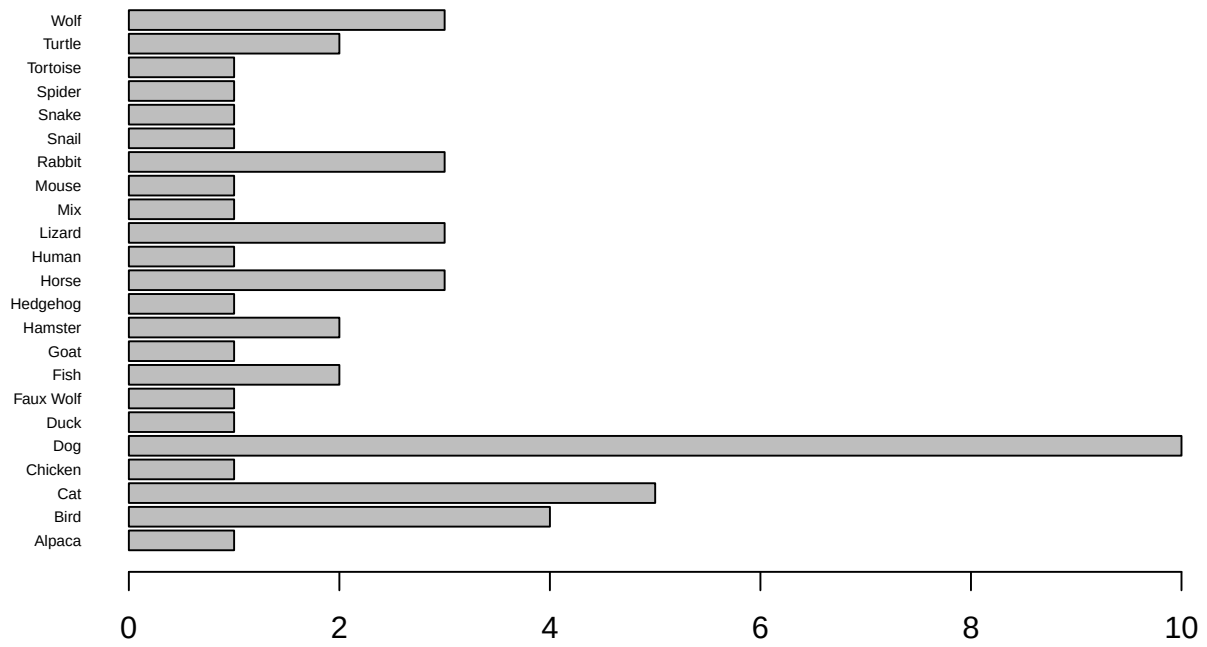
```
# Temperment
val.counts <- data |>
  group_by(Temperment) |>
  count() |>
  arrange(n)

val.counts$Temperment[1] <- "blank"

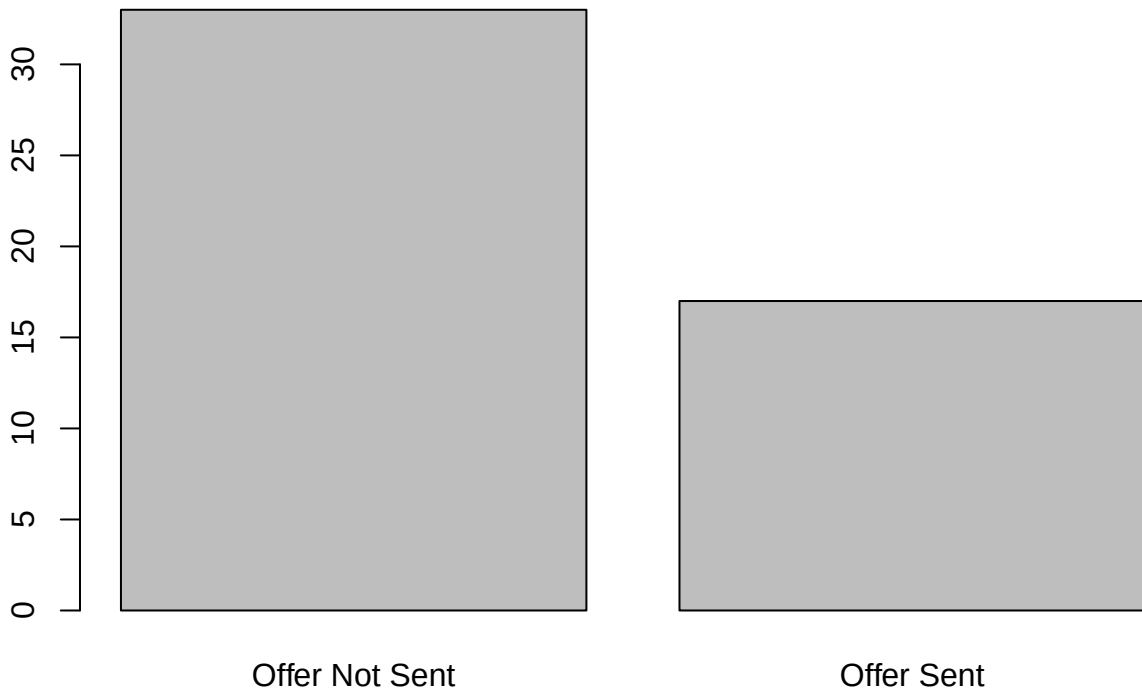
barplot(val.counts$n,
        names.arg = val.counts$Temperment,
        horiz = TRUE,
        xlab = "Temperment",
        las = 1,
        cex.names = .5) # need to make text even smaller to fit everything!
```



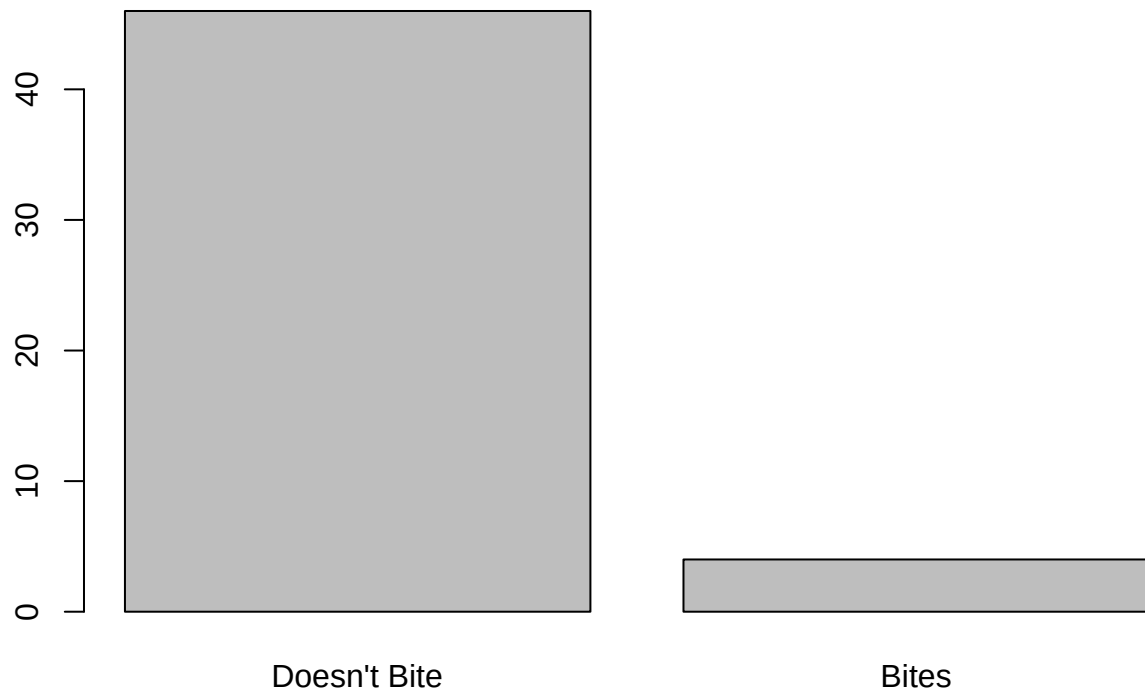
```
# Species
barplot(table(data$Species),
        horiz = TRUE,
        las = 1,
        cex.names = .5) # need to make text even smaller to fit everything!
```



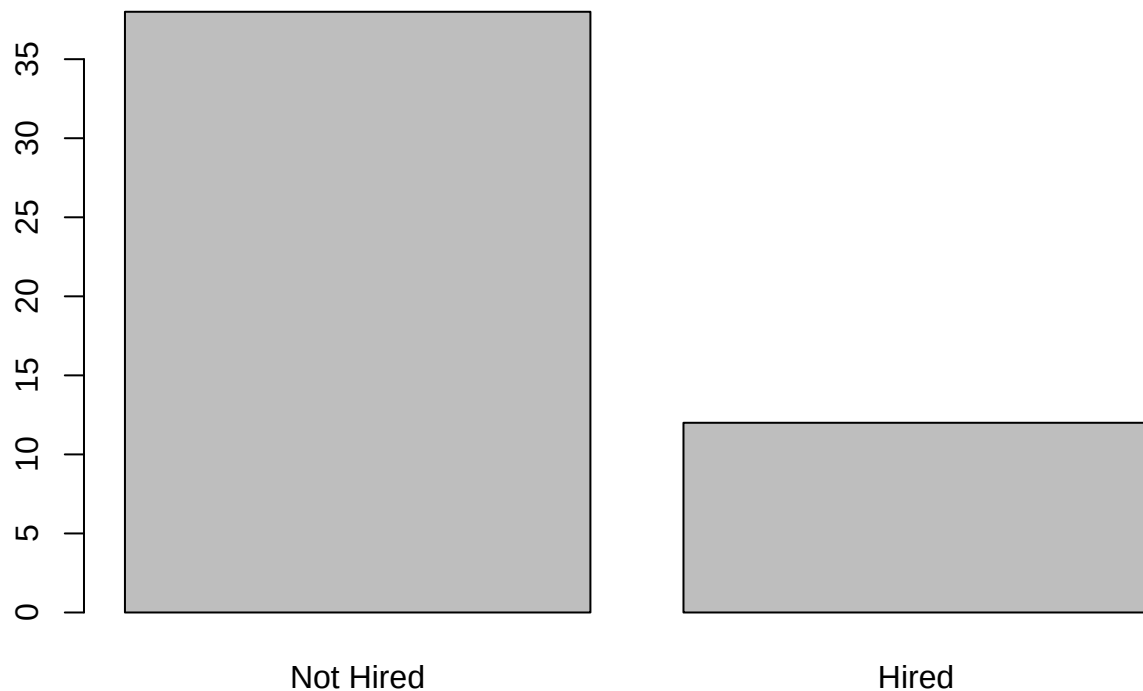
```
# Offer.Sent
barplot(table(data$Offer.Sent))
```



```
# Biter  
barplot(table(data$Biter))
```



```
# Hired  
barplot(table(data$Hired))
```



Dates

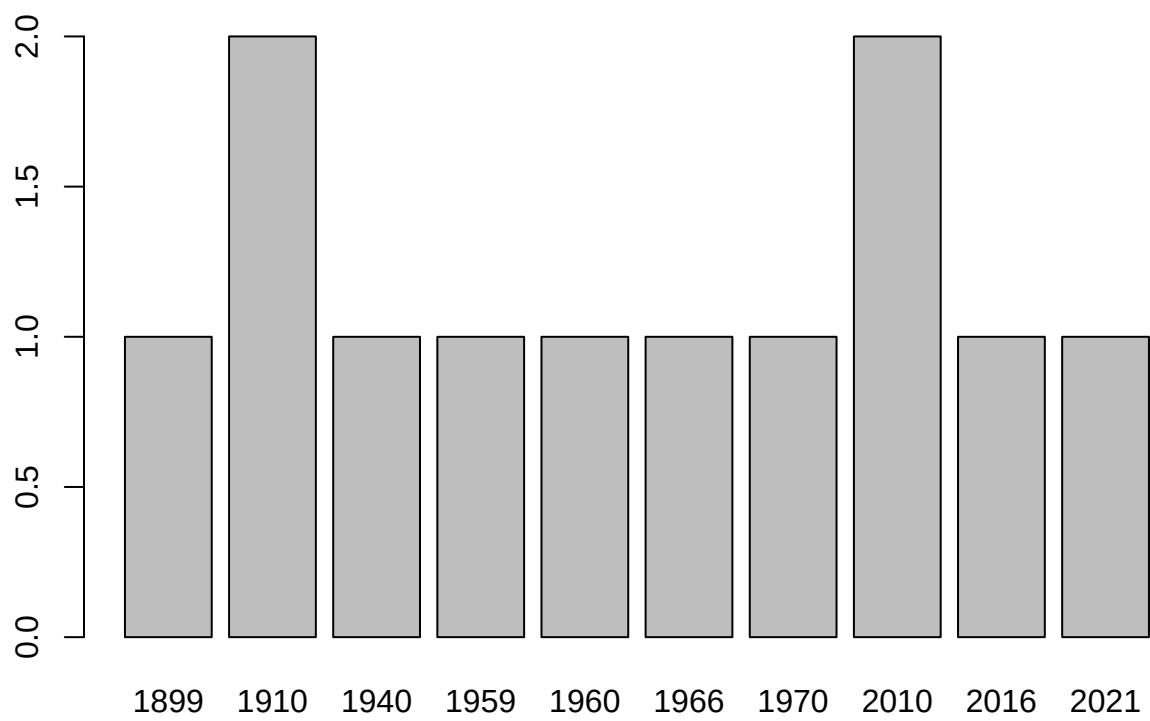
Dates can be visualized as categorical variables by grouping them. Grouping over day, month, year or a combination may be informative.

Dates in our data include:

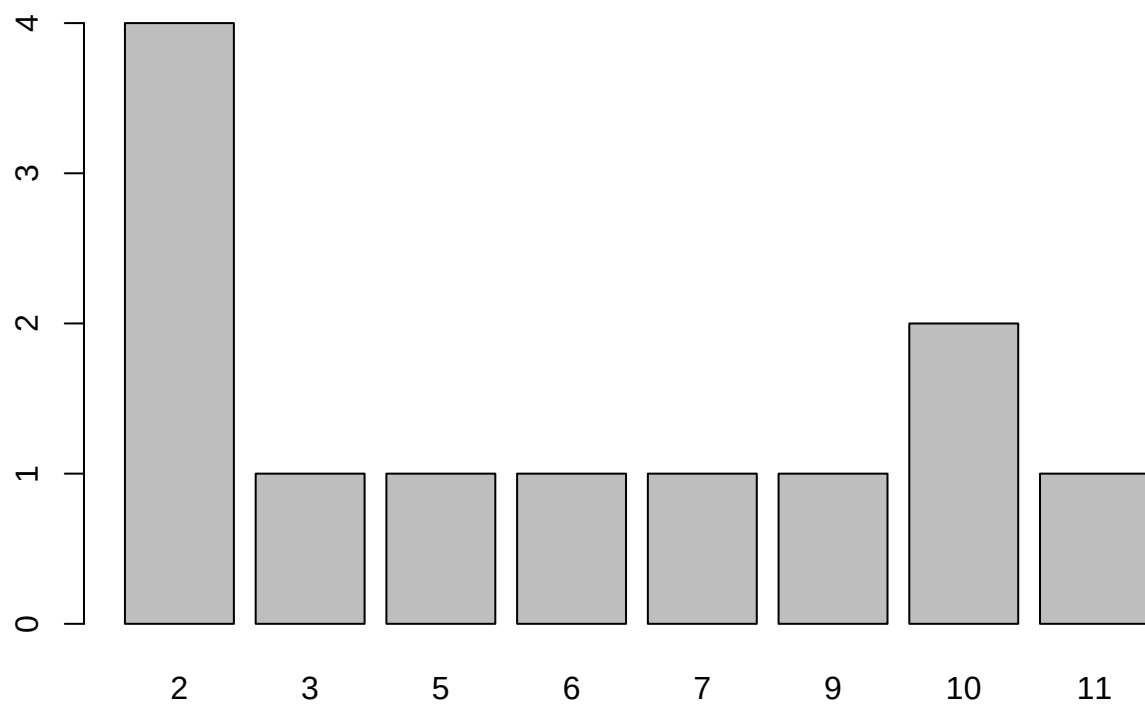
- Interview.Date
- Hire.Date
- Termination.Date

We can use functions like: `year()`, `month()`, `day()` and `wday()` to visualize different timescales. Remember we still have to use `table` to get the counts of the data at that scale.

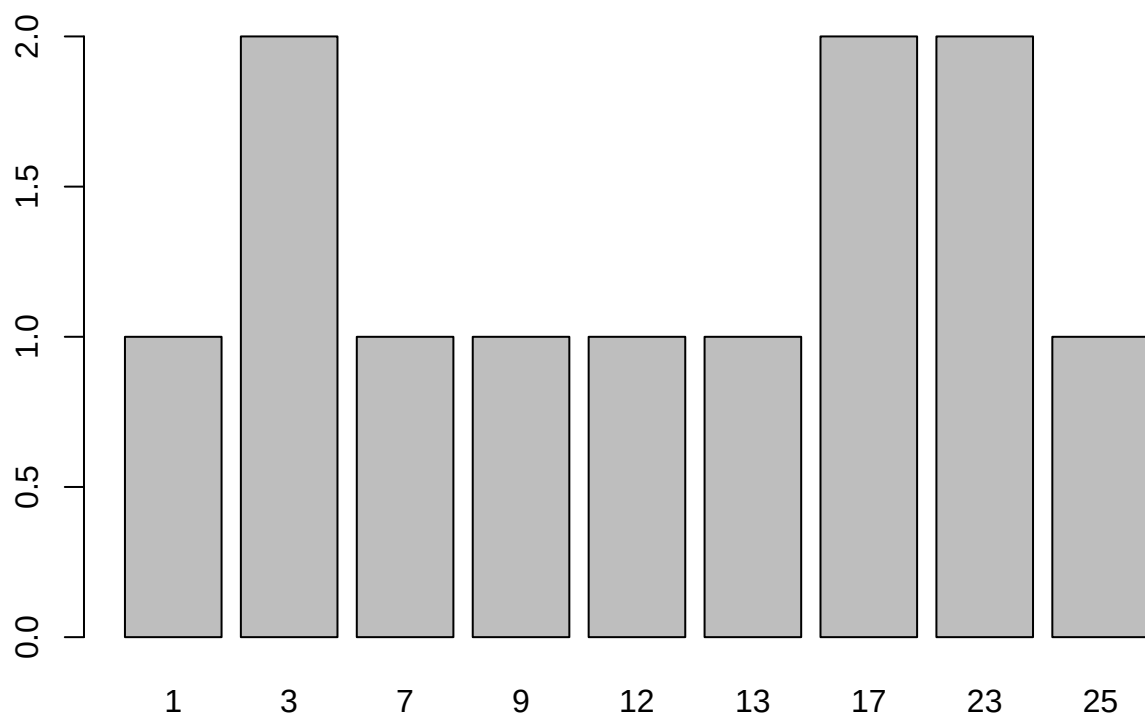
```
# Use year()
cnts <- table(year(data$Hire.Date))
barplot(cnts)
```



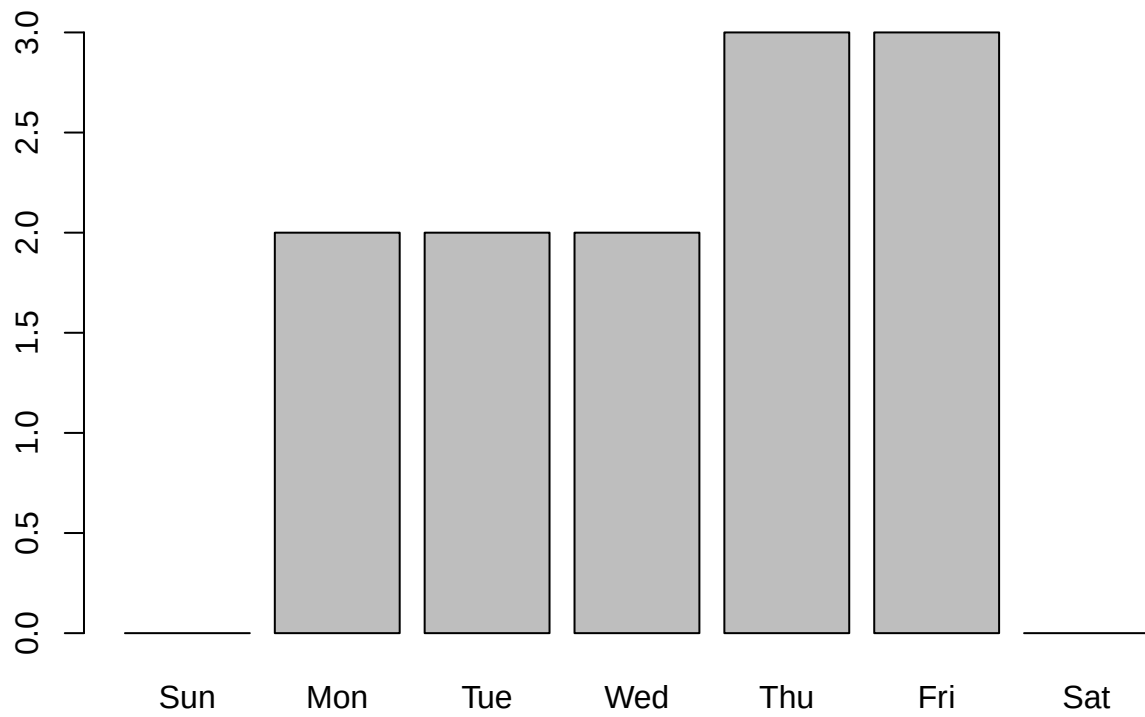
```
# Use month()  
cnts <- table(month(data$Hire.Date))  
barplot(cnts)
```



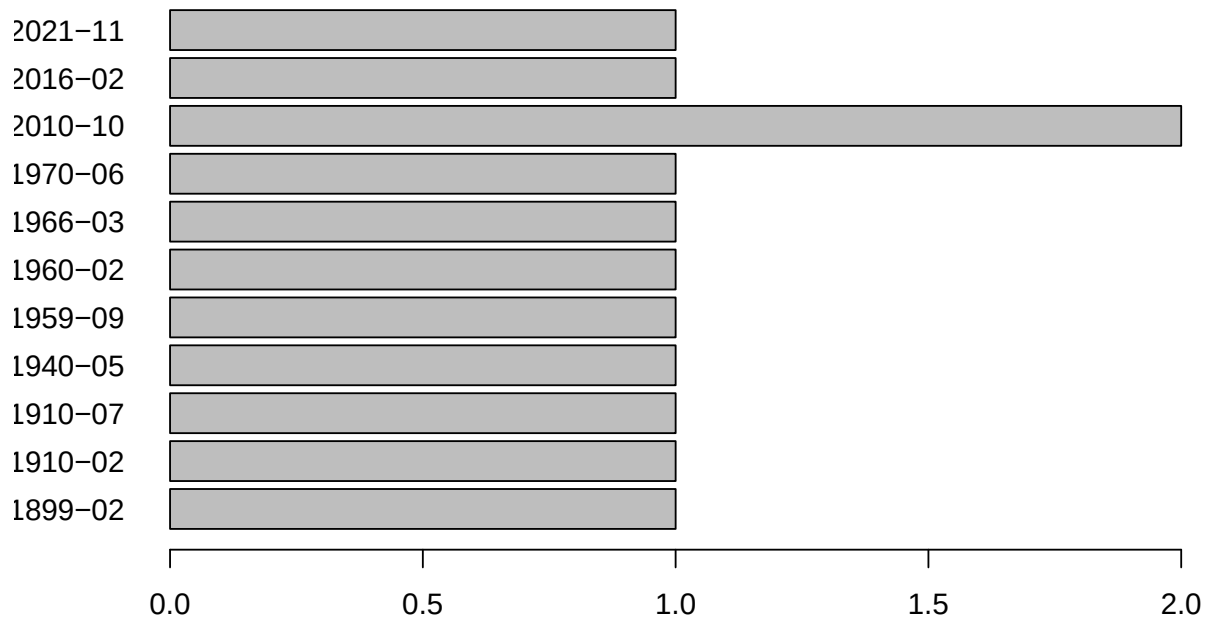
```
# Use day()
cnts <- table(day(data$Hire.Date))
barplot(cnts)
```



```
# Use wday()
cnts <- table(wday(data$Hire.Date, label=TRUE))
barplot(cnts)
```



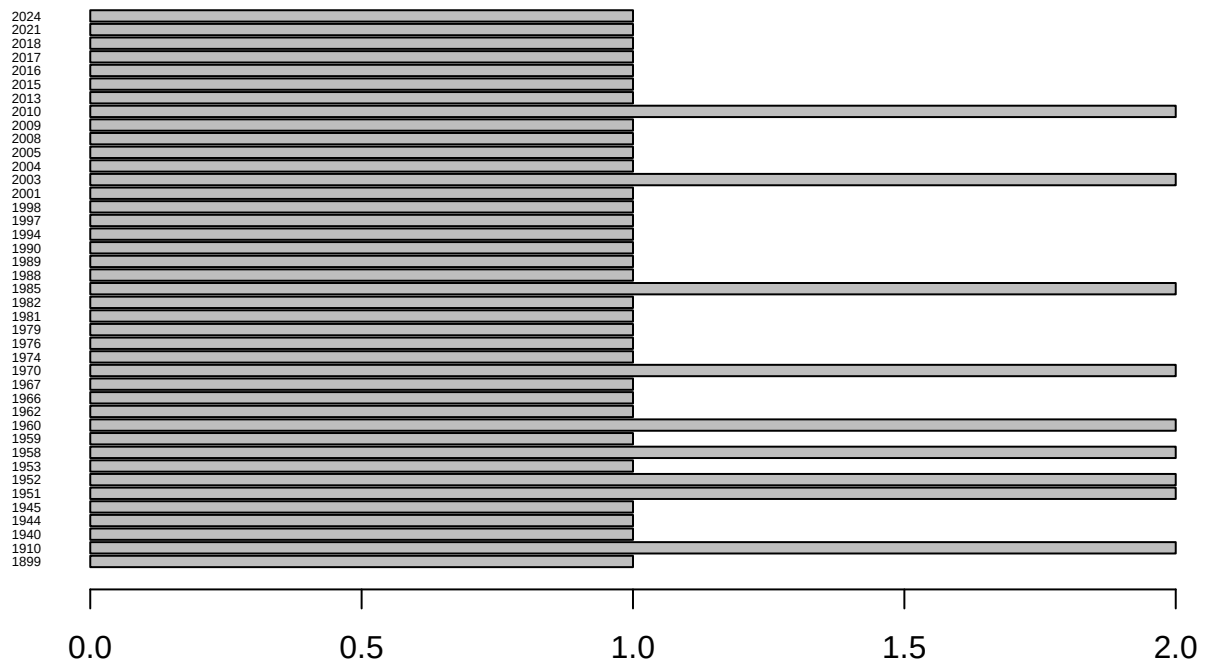
```
# Use format to get YYYY-MM
yr.mnt <- format(data$Hire.Date, "%Y-%m")
cnts <- table(yr.mnt)
barplot(cnts,
        horiz = TRUE,
        las = 1)
```



TRY IT!

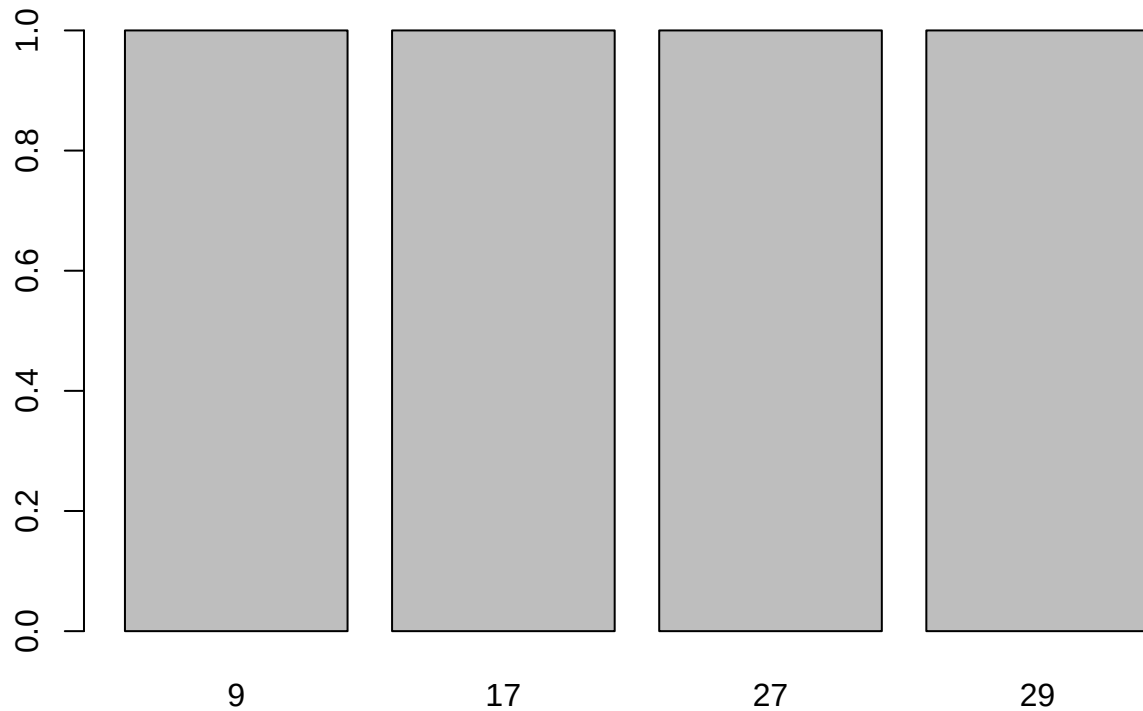
Using the code above make bar plots using the date functions shown above for our other date fields. Are the charts informative?

```
# Interview Date
cnts <- table(year(data$Interview.Date))
barplot(cnts,
        horiz = TRUE,
        las = 1,
        cex.names = .4)
```



```
# Is this chart helpful...not really
# maybe try other functions!

# Termination Date
cnts <- table(day(data$Termination.Date))
barplot(cnts)
```



```
# did you find a helpful chart by trying different date functions?
# I didn't... not enough information in this field.
```

Bivariate Data

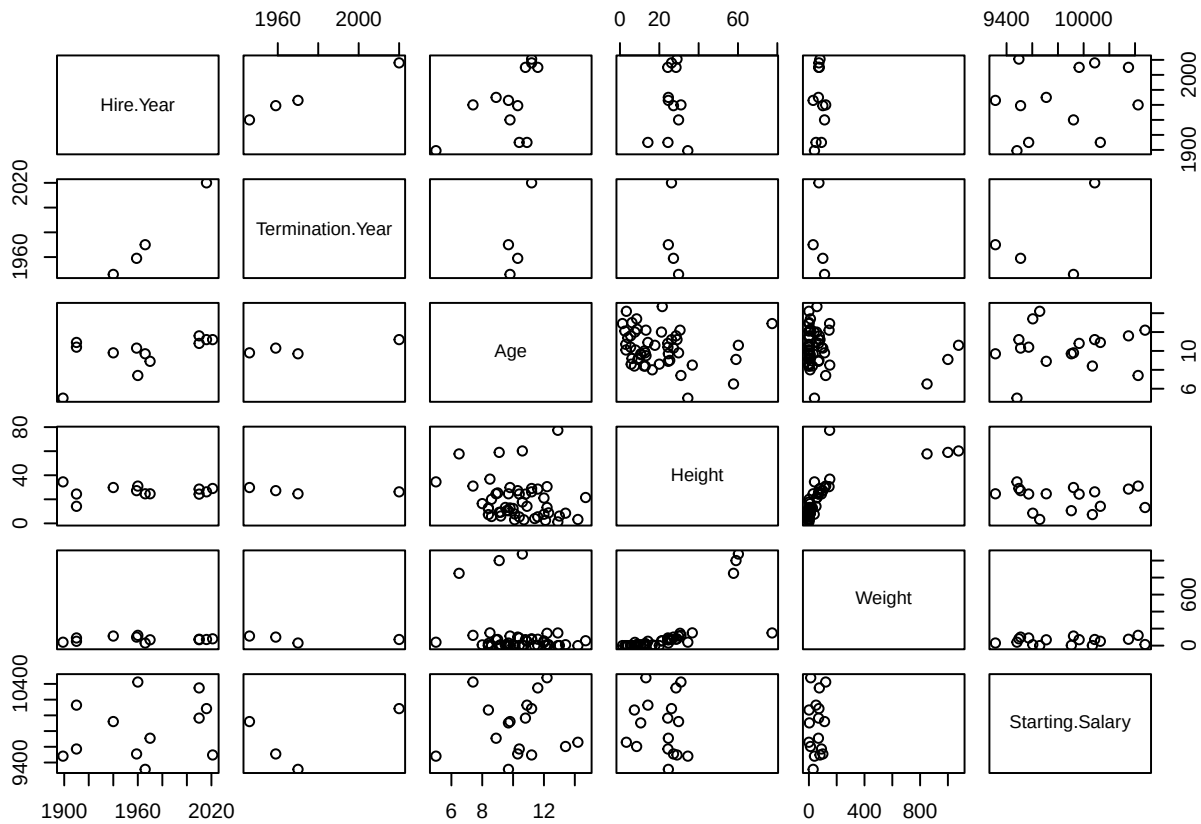
Pair Plots (Numerical Data)

Pair plots are an excellent way to identify interesting patterns between. To use it we must first extract numeric fields.

```
# identify numeric fields in data
num.cols <- data |>
  select(where(is.numeric)) # a smaller df with character columns
names(num.cols)
```

```
[1] "Hire.Year"      "Termination.Year" "Age"             "Height"
[5] "Weight"        "Starting.Salary"
```

```
pairs(num.cols)
```

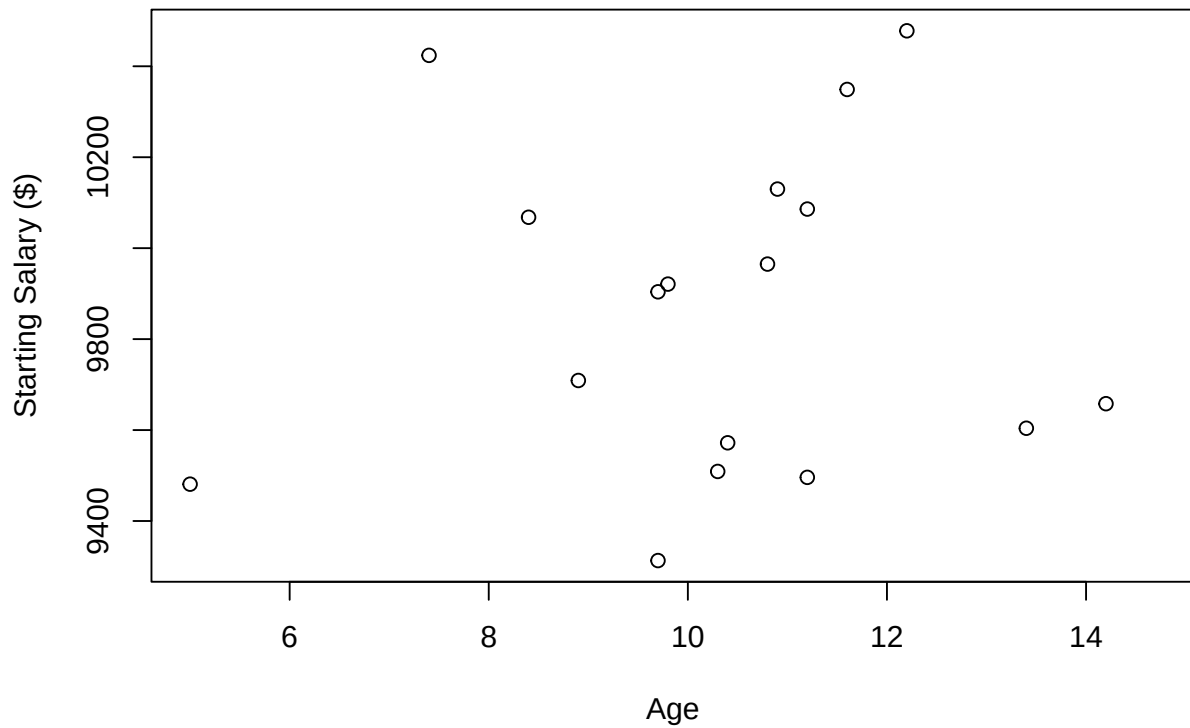


The `pairs()` function in essence provides scatterplots in a grid that visualizes the relationship between every **pair** of numeric values.

Scatter Plots (2 Numerical Variables)

To investigate these relations further we can create a single scatter plot.

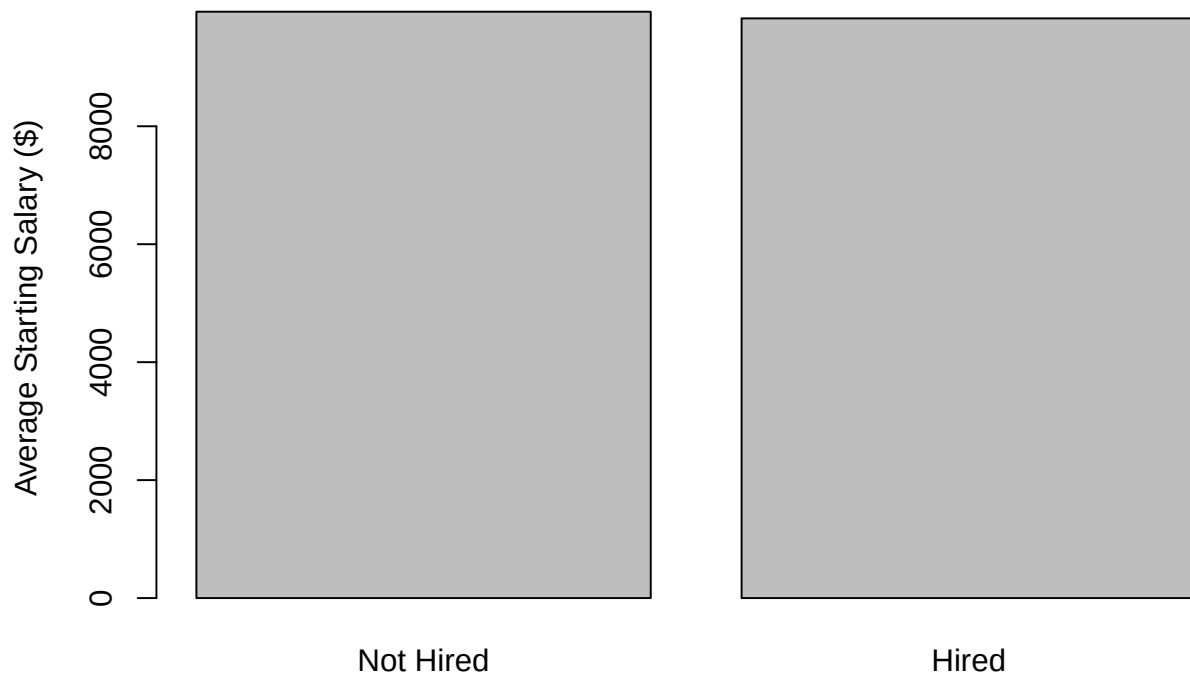
```
plot(data$Age, data$Starting.Salary,
     xlab = "Age",
     ylab = "Starting Salary ($)")
```



Bar charts (1 Categorical & 1 Numeric Variable)

```
data2 <- data |>
  group_by(Hired) |>
  summarise(Salary = mean(Starting.Salary, na.rm = TRUE))

barplot(data2$Salary,
  names.arg = data2$Hired,
  ylab = "Average Starting Salary ($)"
)
```



Grouped Bar Charts (2 Categorical and 1 Numerical)

```
# Create counts table
counts <- table(data$Hired, data$Temperment)
counts
```

		bewildered	calm	cranky	devilishly	clever	distracted	frightened
Not Hired	0	3	0	4		4	5	3
Hired	1	1	1	0		1	0	0

		mouthy	patient	quiet	social	sweet
Not Hired	3	9	2	3	2	
Hired	0	1	1	4	2	

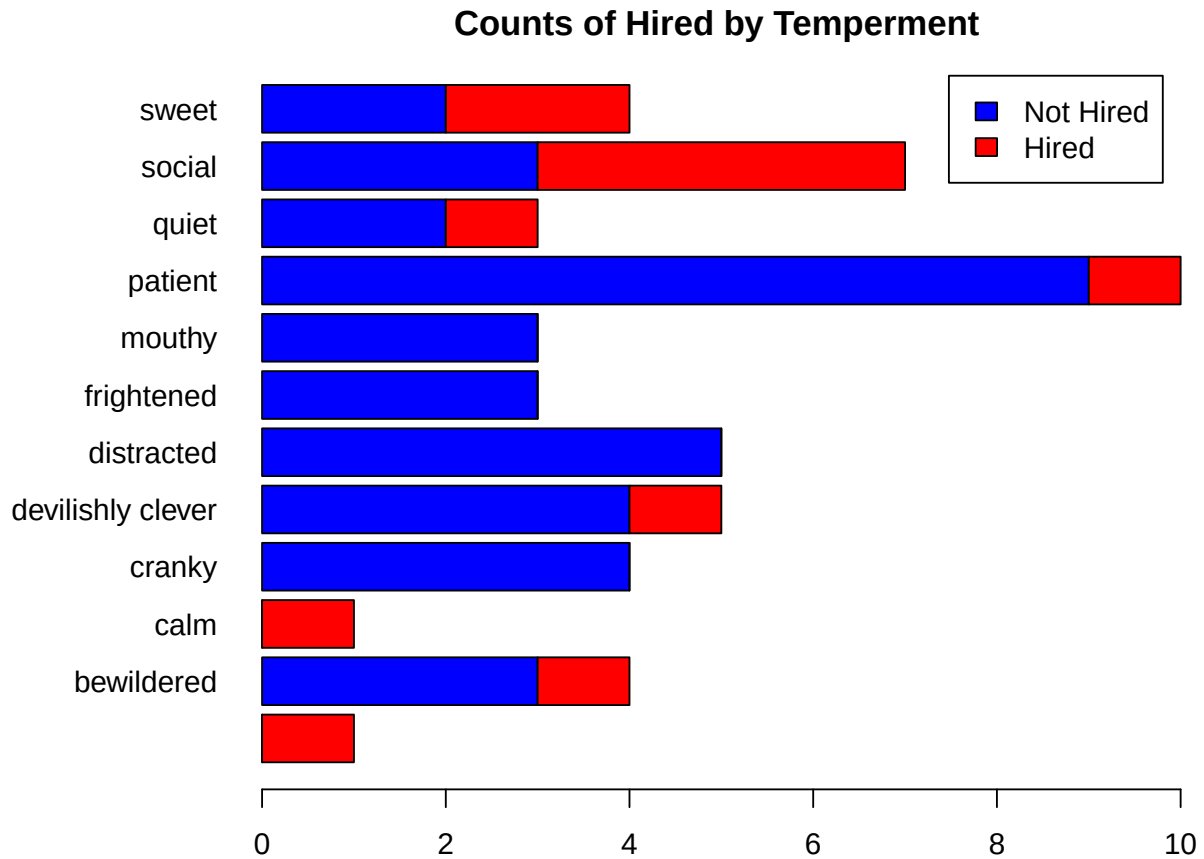
```
# Set mar()
par(mar = c(2.5, 7, 2, 2))

# Generate plot
barplot(counts,
        main = "Counts of Hired by Temperment",
        xlab = "Temperment", col = c("blue", "red"),
```

```

legend = rownames(counts),
horiz = TRUE,
las = 1,
cex.lab = .75)

```



```

# reset mar()
par(mar = c(5.1, 4.1, 4.1, 2.1))

```

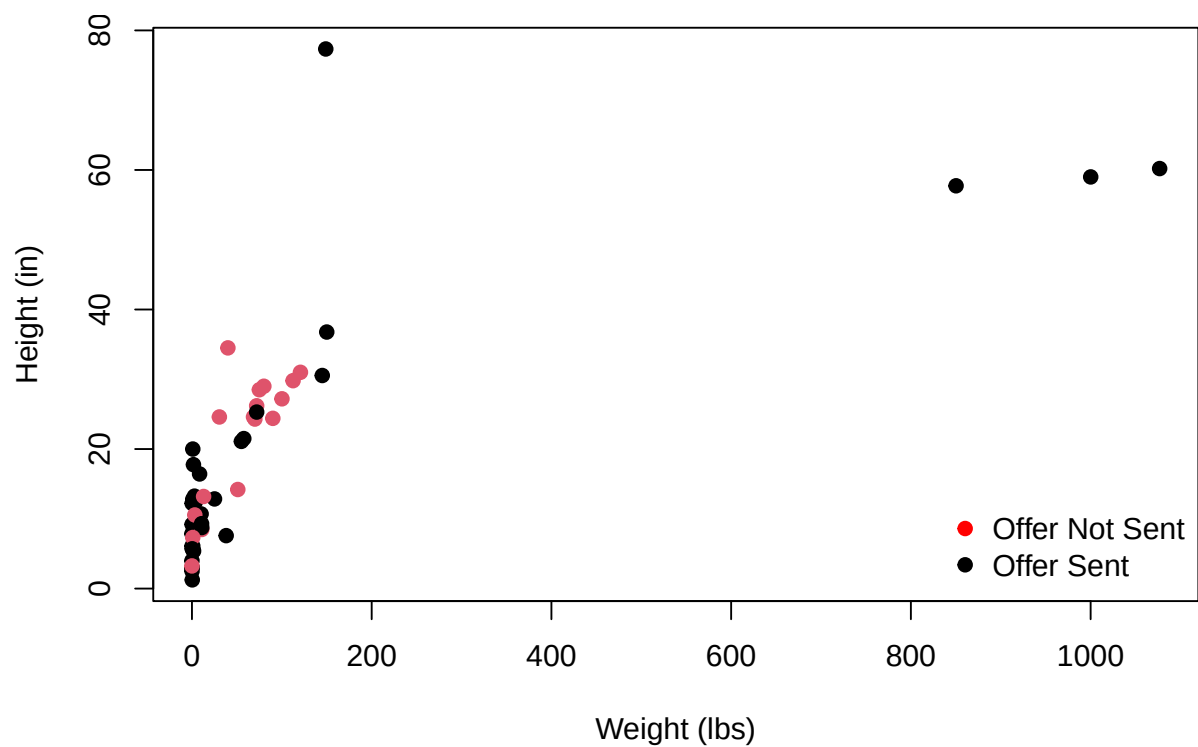
Multivariate Plots

Scatter plot (2 Numerical and 1 Factor/Categorical)

NOTE: Be sure to run the following code using the run button. Otherwise, you will get an annoying error.

```
# Using a factor
plot(data$Weight, data$Height,
     pch = 19,
     col = data$Offer.Sent,
     xlab = "Weight (lbs)",
     ylab = "Height (in)")

legend('bottomright',
     col = c('red', 'black'),
     pch = 19,
     legend = levels(data$Offer.Sent),
     bty = 'n')
```



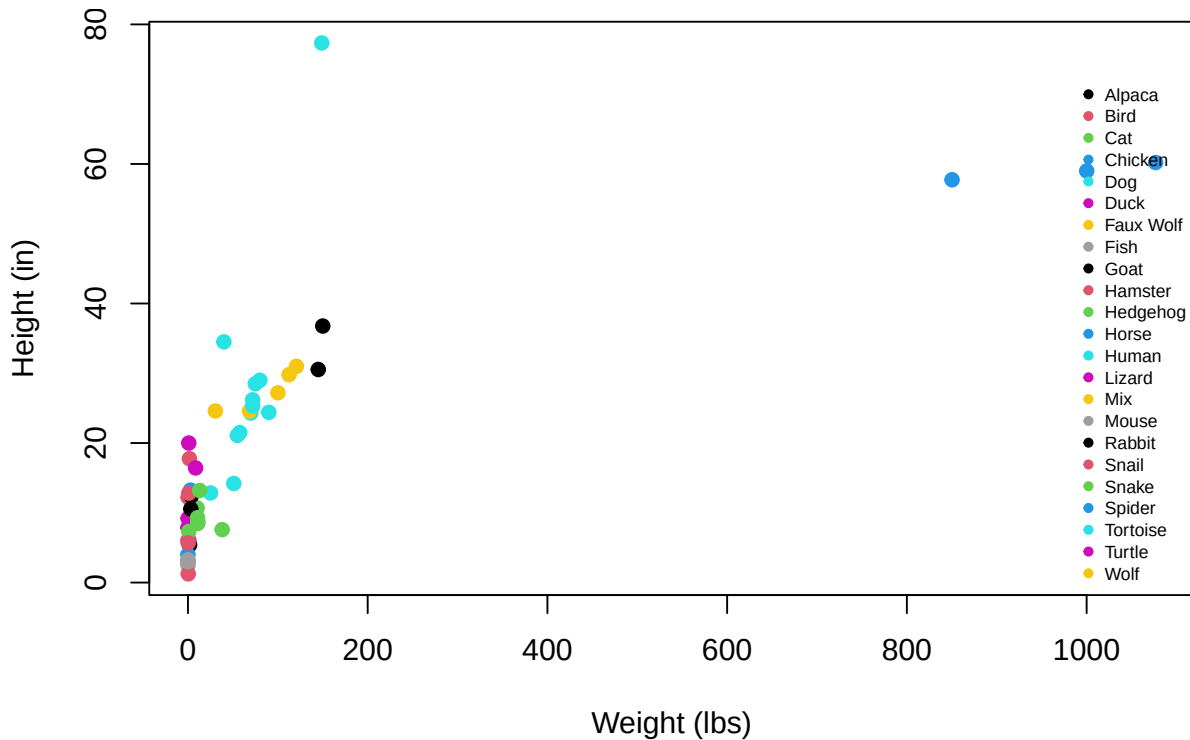
```
# Create plot
plot(data$Weight, data$Height,
     pch = 19,
     col = as.factor(data$Species),
     xlab = "Weight (lbs)",
     ylab = "Height (in)")

legend('bottomright',
```

```

legend = levels(as.factor(data$Species)),
col = as.factor(levels(as.factor(data$Species))),
cex = .6,
bty = 'n',
pch = 19)

```



Bar Chart (1 Numeric and 2 Factor/Categorical)

We can generate barplots for multivariate data by using grouped or stacked bar charts. Note that making this work with data frames is more difficult than with previous plots. This is when you start considering to move to `ggplot2`.

```

# generate a new data frame data by grouping data by two categorical variables
# and summarizing over a continuous variable
data2 <- data |>
  group_by(Temperment, Hired) |>
  summarise(Salary = mean(Starting.Salary, na.rm = TRUE)) |>
  drop_na() |>
  filter(Temperment != "") |>
  pivot_wider(

```

```

names_from = Temperment, # The field with the most values.
values_from = Salary,
values_fill = 0
)

```

`summarise()` has regrouped the output.

- i Summaries were computed grouped by Temperment and Hired.
- i Output is grouped by Temperment.
- i Use `summarise(.groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(Temperment, Hired))` for per-operation grouping (`?dplyr::dplyr\_by`) instead.

```

# barplot() will work best with a matrix that has column and row names
# we will pull the row names for the first column of data.

```

```

# save row names in variable
r.names <- data2$Hired

# save the Salary values only as a matrix
data.matrix <- as.matrix(data2[, 2:9])

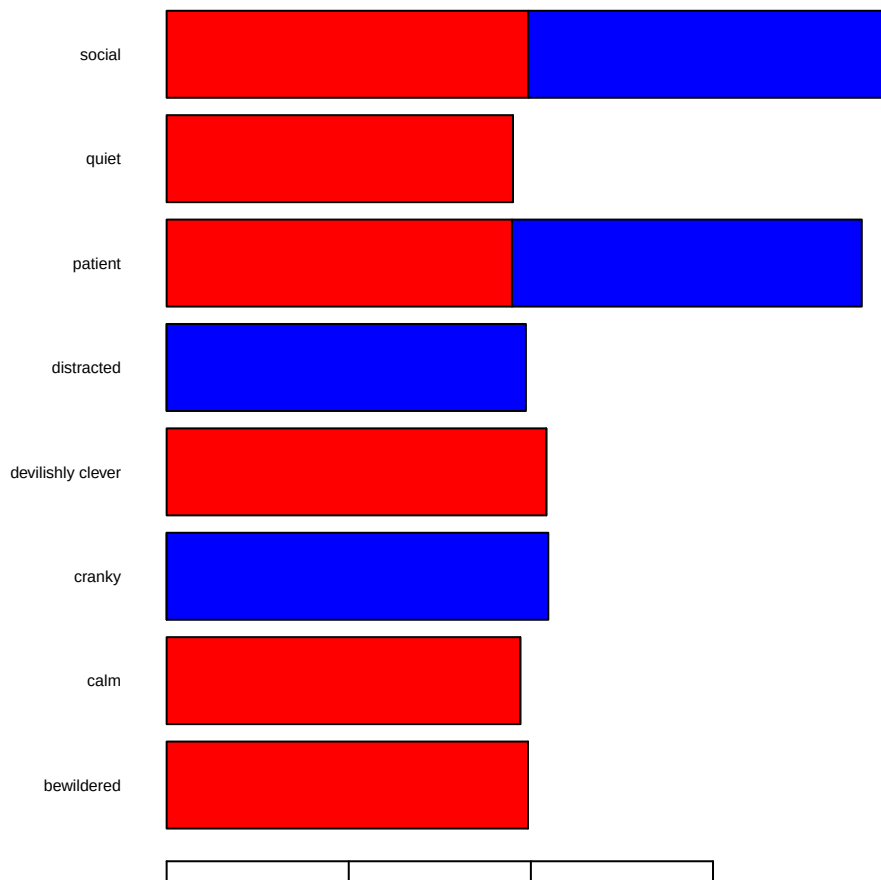
# add row names to the new matrix
rownames(data.matrix) <- r.names

# use data to create bar chart

# Set mar()
par(mar = c(1, 8, 1, 8))

# Generate Plot
barplot(data.matrix,
        col = c("red", "blue"),
        horiz = TRUE,
        las = 1,
        cex.names = .5)

```



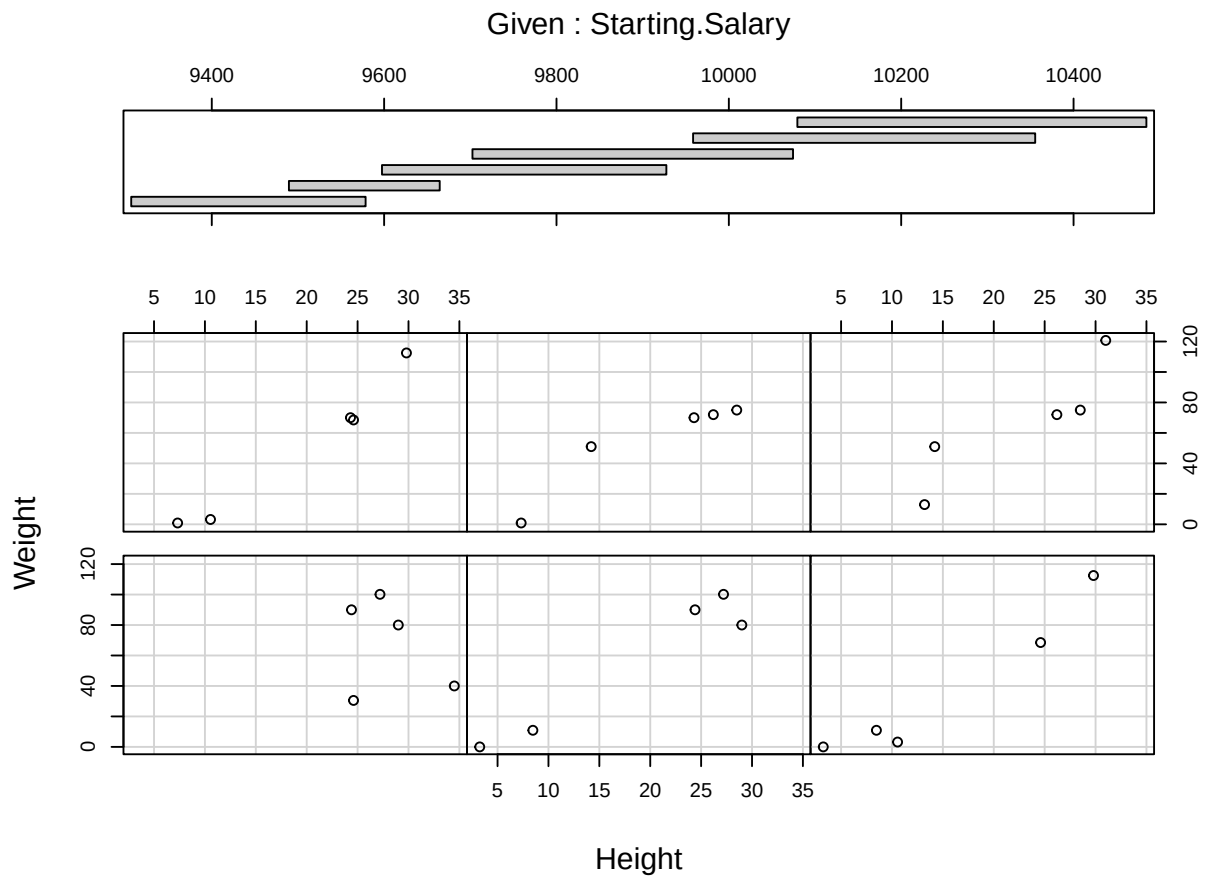
```
# reset mar()
par(mar = c(5.1, 4.1, 4.1, 2.1))
```

Coplots

Coplots, or conditioning plots,

```
# extract data for coplot
coplot.data <- data |>
  drop_na(Weight, Height, Starting.Salary) |>
  select(Weight, Height, Starting.Salary) |>
  arrange(Starting.Salary, Weight, Height)

# generate coplot
coplot(Weight ~ Height | Starting.Salary, data = coplot.data)
```



The top graph shows ranges of salary for which the plots below are partitioned. The plots reflect the ranges from bottom left to top right. The image below shows which plots the salary ranges are associated with. Note that you can use categorical variables and can add another conditioning variable as well. See more information and examples at [R coplots](#).

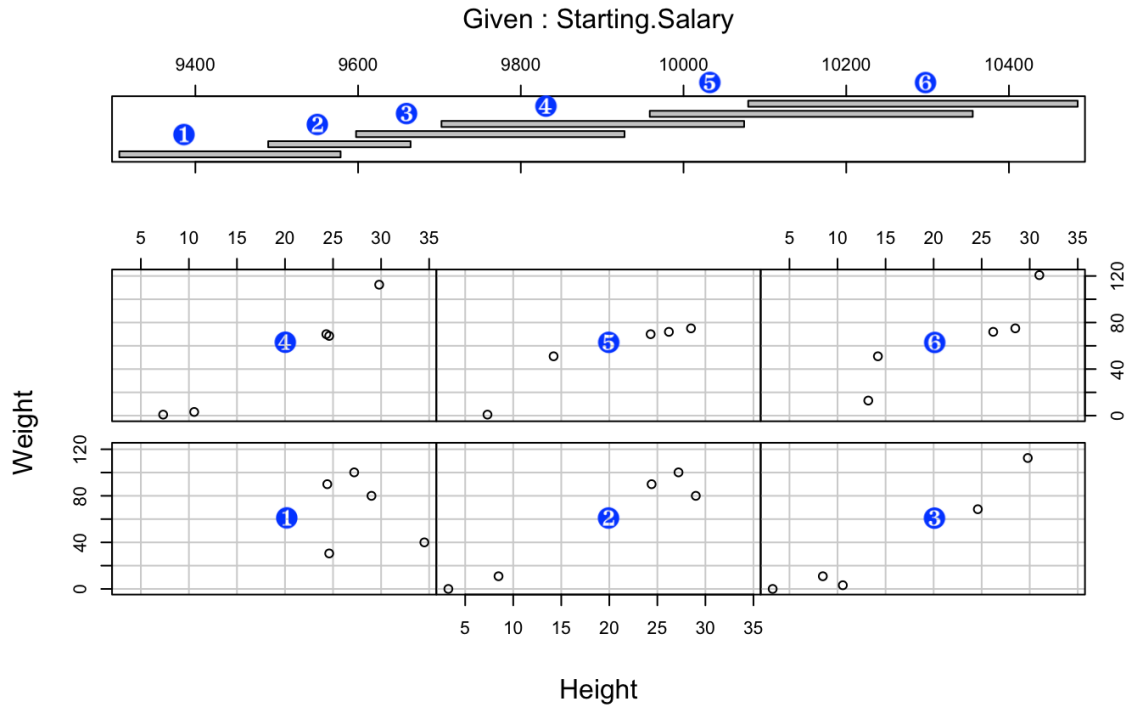
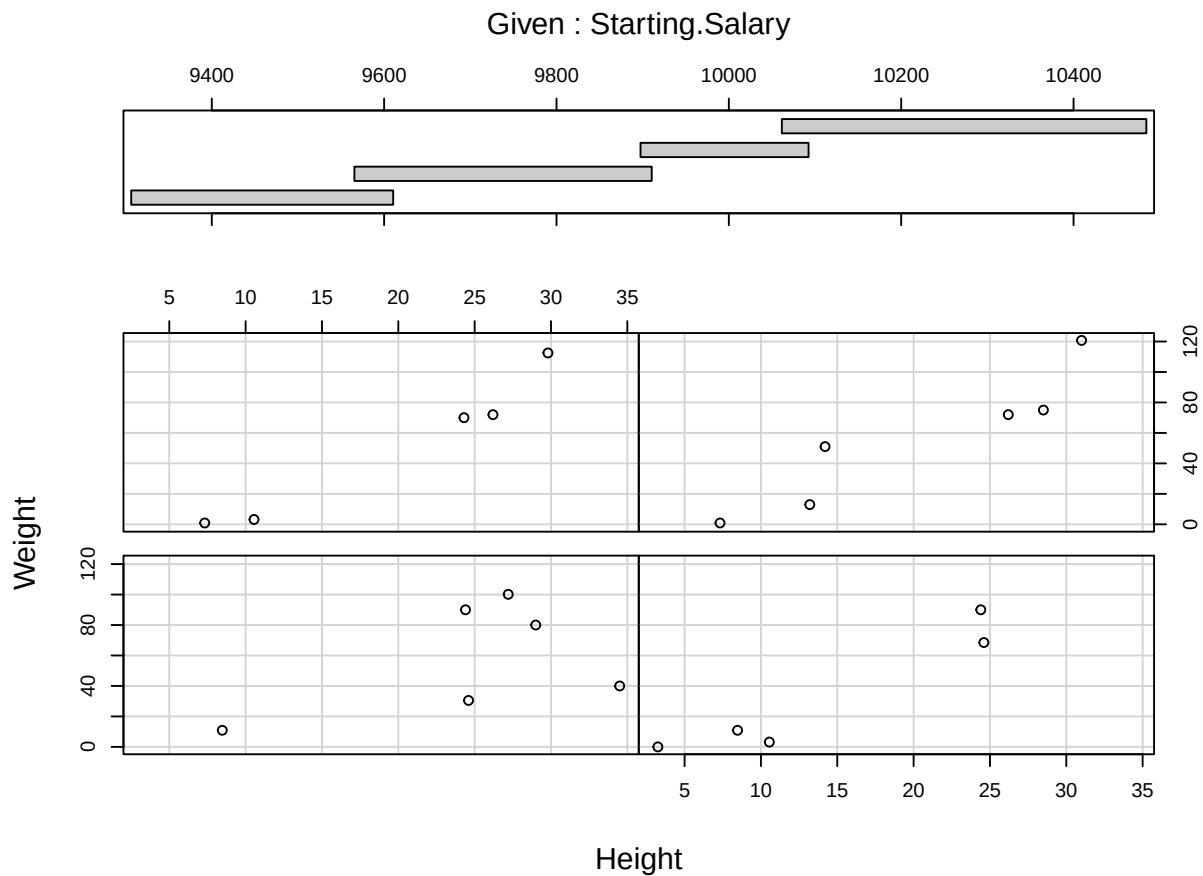


Figure 2: Conditional Plot Layout

The `co.intervals()` function lets us control the number of intervals and the overlap between intervals for continuous variables. Below, we reduce the number of intervals to 4 and restrict the overlap to 33%.

```
# revise intervals for Starting Salary
given.salary <- co.intervals(coplot.data$Starting.Salary,
                             number=4, overlap=.33)

# regenerate plot using new intervals
coplot(Weight ~ Height | Starting.Salary, data = coplot.data,
       given.values = given.salary)
```



Next Steps

We have explored the data quite a bit. Next steps would be to identify one or two relationships to explore further. This may involve subsetting the data further or using simple models like linear (continuous, fairly normal response) or logistic regression (binary/boolean response).

What relationships or trends do you think might be interesting?

Based on the data, **what job do you think Barbara has at NC State?**

TRY IT ON YOUR OWN!

Using the code above try exploring another data set. Explore the data to determine what the data represents and to find interesting relationships between the variables, if any.

```
new.data <- read.csv("../data/EDA_Data_File_2.csv")
```

Links to additional resources

R

[R for Data Science](#)

[Introduction to R](#)

Base R Plots

[Intro2R](#)

[STHDA](#)